INSTITUTO TECNOLÓGICO DE LA PAZ
DIVISIÓN DE ESTUDIOS DE POSGRADO E INVESTIGACIÓN
MAESTRÍA EN SISTEMAS COMPUTACIONALES

LOGÍSTICA DE ÚLTIMA MILLA EN REDES VIALES REALES: UN ENFOQUE REPRODUCIBLE PARA EL MDRP EN LA PAZ, B.C.S.

TESIS

QUE PARA OBTENER EL GRADO DE
MAESTRO EN SISTEMAS COMPUTACIONALES

PRESENTA:
ALAN JOSÉ MARCOS SÁNCHEZ MARTÍNEZ

DIRECTOR DE TESIS:
DR. MARCO ANTONIO CASTRO LIERA

LA PAZ, BAJA CALIFORNIA SUR, MÉXICO, JUNIO 2026.

Índice general

1. Introducción **1**

1. 1. Antecedentes 1

1. 1.1. De TSP a VRP 1

1. 1.2. VRP clásico y extensiones con restricciones 2

1. 1.3. Última milla y contexto urbano 3

1. 1.4. Enrutamiento dinámico: DVRP y DARP 4

1. 1.5. DVRP: Definición y características fundamentales 4

1. 1.6. DARP: Del transporte de pasajeros a la entrega de bienes 5

1. 1.7. El valor de la información futura y el enfoque de Horizonte Rodante . . . 6

1. 1.8. MDRP y políticas de despacho 7

1. 1.9. Características compartidas con DVRP y DARP 7

1. 1.10. Definición y características del MDRP 7

1. 1.11. Enrutamiento realista en redes viales 8

1. 2. Planteamiento del problema 9

1. 2.1. El problema de decisión: coordinar o reaccionar 9

1. 2.2. La brecha: ciudades medias y redes viales reales 9

1. 2.3. El caso de La Paz, B.C.S. 10

1. 2.4. Preguntas de investigación 10

1. 3. Objetivos 11

1. 3.1. Objetivo general 11

1. 3.2. Objetivos específicos 11

1. 4. Hipótesis 12

1. 5. Justificación 12

1. 5.1. Relevancia Científica 12
1. 5.2. Aporte Tecnológico y Ciencia Abierta 13
1. 5.3. Impacto Económico y Social 13
1. 5.4. Delimitación frente a métodos de Aprendizaje Profundo 13
1. 6. Alcance y limitaciones 14
1. 6.1. Alcance Geográfico y de Datos 14
1. 6.2. Alcance Funcional y Algorítmico 14
1. 6.3. Alcance Técnico: Reproducibilidad e Infraestructura 15
1. 6.4. Supuestos y Limitaciones 16

2. Marco teórico **19**
2. 1. Formulación matemática del MDRP 19
2. 1.1. Conjuntos y entidades 19
2. 1.2. Función objetivo 20
2. 1.3. Suposiciones estructurales 21
2. 1.4. Modelo de compensación 21
2. 1.5. Métricas de desempeño 22
2. 1.6. Variantes del entorno operativo 22
2. 1.7. Clasificación del MDRP en la literatura 23
2. 2. Heurística de horizonte rodante 23
2. 2.1. Tamaño objetivo de agrupamiento 24
2. 2.2. Generación de agrupamientos 25
2. 2.3. Esquema de prioridad 26
2. 2.4. Modelo de asignación lineal 26
2. 2.5. Estrategia de compromiso en dos etapas 27
2. 3. Enrutamiento sobre grafos viales 28
2. 3.1. OpenStreetMap como modelo de datos geográficos 28
2. 3.2. *Contraction Hierarchies* 29
2. 3.3. OSRM: motor de enrutamiento 30
2. 4. Generación estocástica de demanda y métricas 31
2. 4.1. Proceso de Poisson no homogéneo 31

2. 4.2. Función de intensidad bimodal 32
2. 4.3. Métricas de desempeño: marco de evaluación 33

3. Desarrollo **35**

3. 1. Arquitectura del sistema 35
3. 1.1. Capa de enrutamiento: OSRM contenerizado 35
3. 1.2. Capa de simulación: estructura de módulos Python 39
3. 1.3. Capa de análisis: generación de resultados 41
3. 2. Generación de instancias sintéticas 41
3. 2.1. Generacion temporal de órdenes 42
3. 2.2. Tiempo de preparación 42
3. 2.3. Distribución geoespacial 43
3. 2.4. Dimensionamiento automático de flota 44
3. 3. Motor de simulación 45
3. 3.1. Ciclo de simulación por épocas 45
3. 3.2. Modelos de datos: órdenes, couriers y restaurantes 46
3. 3.3. Máquina de estados de una orden 47
3. 3.4. Temporización de rutas 48
3. 3.5. Integración con OSRM 48
3. 4. Política FCFS (*baseline*) 49
3. 4.1. Lógica de asignación 49
3. 4.2. Características y limitaciones 50
3. 5. Política *Rolling Horizon* 51
3. 5.1. Cálculo del tamaño objetivo de agrupamiento 51
3. 5.2. Generación de agrupamientos por restaurante 52
3. 5.3. Clasificación por prioridad y asignación bipartita 53
3. 5.4. Compromiso en dos etapas 54
3. 5.5. Resumen del flujo RH por época 55
3. 6. Diseño experimental 55
3. 6.1. Variable de tratamiento: saturación de flota 56
3. 6.2. Escenarios 56

3. 6.3. Comparación pareada 56
3. 6.4. Plan de análisis estadístico 57
3. 6.5. Escenario de control euclidiano 57
3. 6.6. Replicación y análisis de sensibilidad 58
3. 6.7. Almacenamiento de resultados 58
3. 7. Métricas de evaluación 58
3. 7.1. Calidad de servicio 59
3. 7.2. Cobertura 59
3. 7.3. Eficiencia operativa 59
3. 7.4. Eficiencia de agrupamiento 60
3. 7.5. Costo 60
3. 7.6. Resumen de indicadores 61

4. Resultados **62**

4. 1. Configuración experimental 62
4. 2. Calidad de servicio 63
4. 2.1. Análisis del *Click-to-Door* 63
4. 2.2. *Ready-to-Pickup* y exceso sobre el objetivo 65
4. 3. Eficiencia operativa 66
4. 3.1. Productividad y utilización 66
4. 3.2. Distancia y enrutamiento 67
4. 4. Agrupamiento y costos 67
4. 4.1. Distribución de agrupamientos 67
4. 4.2. Modelo de compensación 68
4. 5. Análisis estadístico 68
4. 5.1. Diseño inferencial 68
4. 5.2. Resultados 69
4. 5.3. Discusión por regímenes 70
4. 6. Análisis de robustez: orden de las fases del ciclo 71
4. 7. Efecto de la topología vial 73
4. 7.1. Circuidad de la red de La Paz 73

4. 7.2. Comparación con el escenario de control euclidiano 73
4. 8. Desempeño computacional 74
4. 9. Comparación con la literatura y limitaciones 75
4. 9.1. Comparación con la literatura 75
4. 9.2. Limitaciones de los resultados 76

5. Conclusiones **78**
5. 1. Respuesta a las preguntas de investigación 78
5. 2. Verificación de objetivos 79
5. 2.1. Objetivo general 79
5. 2.2. Objetivos específicos 79
5. 3. Verificación de la hipótesis 80
5. 4. Contribuciones principales 81
5. 5. Trabajo futuro 82

Bibliografía **83**

Índice de figuras

3.1. Arquitectura de tres capas del entorno de simulación MDRP-BCS. 36
3.2. Flujo de la política *Rolling Horizon* en cada época de asignación. 55

4.1. Distribución del *Click-to-Door* por escenario y política. La línea roja punteada indica el objetivo SLA de $\tau = 40$ min. 65
4.2. Distribución del tamaño de agrupamiento bajo RH por escenario. 68
4.3. Evolución de KPIs principales en función de la saturación. Las curvas de CtD se cruzan entre los ratios 18 y 26. 71

Índice de tablas

3.1. Módulos del simulador y su responsabilidad. 39
3.2. Parámetros de configuración del simulador (`config.py`). 40
3.3. Escenarios experimentales de saturación ($|\mathcal{O}| \approx 1\,038$ pedidos). 56
3.4. Indicadores clave de desempeño (KPIs) evaluados. 61

4.1. Escenarios de saturación de flota (rango de $|C|$ entre las diez semillas). 63
4.2. Indicadores de calidad de servicio: promedio de 10 réplicas. $\Delta\,\%$ es el cambio relativo de RH respecto a FCFS; Δpp, la diferencia en puntos porcentuales (calculada con valores sin redondear). 64
4.3. *Ready-to-Pickup* y *CTD Overage*: promedio de 10 réplicas. 65
4.4. Indicadores de eficiencia operativa: promedio de 10 réplicas. 66
4.5. Indicadores de agrupamiento y costos: promedio de 10 réplicas. *Avg Bundle* es el tamaño del agrupamiento en que viaja cada pedido (media ponderada por pedido); Órd./viaje es el número medio de órdenes por ruta. 67
4.6. Wilcoxon *signed-rank* pareado a nivel semilla ($n = 10$): *Click-to-Door*, *Ready-to-Door* y SLA *Compliance*. 69
4.7. Robustez frente al orden de las fases del ciclo: campaña principal (actualización después de asignar) y variante con actualización antes de asignar. Media de 10 semillas; $\Delta\mathrm{CtD} = \mathrm{CtD}_{\mathrm{FCFS}} - \mathrm{CtD}_{\mathrm{RH}}$ (min), cambio relativo del CtD de RH respecto a FCFS (%, media por semilla), $\Delta\mathrm{SLA} = \mathrm{SLA}_{\mathrm{RH}} - \mathrm{SLA}_{\mathrm{FCFS}}$ (pp) y porcentaje de pedidos que FCFS no entrega. 72
4.8. Ventaja de RH sobre FCFS con la red vial real (OSRM) y con distancias en línea recta (Euclid.): media de 10 semillas. $\Delta\mathrm{CtD} = \mathrm{CtD}_{\mathrm{FCFS}} - \mathrm{CtD}_{\mathrm{RH}}$; $\Delta\mathrm{SLA} = \mathrm{SLA}_{\mathrm{RH}} - \mathrm{SLA}_{\mathrm{FCFS}}$. 74

4.9. Ventaja de RH en CtD ($\Delta\text{CtD} = \text{CtD}_{\text{FCFS}} - \text{CtD}_{\text{RH}}$, min) con la red vial real, con el control euclidiano a la velocidad media (651 m/min) y con el control euclidiano escalado por κ (472 m/min): media de 10 semillas. La última columna es la fracción de la diferencia OSRM–control explicada por el escalamiento, (Euclid.$\times\kappa$ − Euclid.)/(OSRM − Euclid.). 75
4.10. Tiempo de cómputo por día simulado (semilla 2025, 1 038 órdenes) y memoria máxima del proceso, con OSRM local. 75

Capítulo 1

Introducción

1. 1. Antecedentes

1. 1.1. De TSP a VRP

El Problema del Vendedor Viajero (TSP, por sus siglas en inglés) es uno de los problemas de optimización combinatoria más estudiados y fundamentales en la investigación de operaciones. En su forma más simple, el problema busca determinar el recorrido más corto posible que visita un conjunto de ciudades exactamente una vez, comenzando y terminando en la misma ciudad.

A pesar de su sencilla definición, el TSP es NP-duro [1]: no se conoce un algoritmo que lo resuelva en tiempo polinomial, y el número de recorridos posibles crece factorialmente con el número de ciudades, de modo que las instancias de gran tamaño resultan computacionalmente intratables para métodos exactos.

El TSP sirve como pilar para una clase más amplia y práctica de problemas de logística, que se conoce como el Problema de Enrutamiento de Vehículos (VRP). La transición conceptual del TSP al VRP ocurrió con el trabajo seminal de Dantzig and Ramser [2], quienes formularon *The Truck Dispatching Problem*. Este trabajo generalizó el TSP al considerar una flota de vehículos con capacidad limitada, con el objetivo de servir a un conjunto de clientes desde un depósito central. Esta generalización marcó el nacimiento del VRP, que busca minimizar el costo total de las rutas (usualmente en términos de distancia o tiempo) para una flota de vehículos que atiende a un conjunto de clientes con demandas conocidas.

Al igual que el TSP, el VRP es NP-duro [1], pero su estructura es significativamente más

rica y compleja. La complejidad no solo reside en encontrar las secuencias óptimas de visitas (como en el TSP), sino también en asignar clientes a vehículos y rutas de manera eficiente [3]. A lo largo de cinco décadas, el VRP ha evolucionado para incorporar una gran variedad de restricciones del mundo real, dando lugar a los llamados *Rich VRPs* [3]. Estas extensiones incluyen, entre otras:

1. 1.2. VRP clásico y extensiones con restricciones

- **Capacidades de los vehículos (CVRP)**: Donde cada vehículo tiene una capacidad máxima de carga que no puede ser excedida. El CVRP introduce la decisión adicional de no solo determinar la secuencia de visitas, sino también decidir cuánto llevar en cada viaje, lo que aumenta significativamente la complejidad computacional del problema. Cada vehículo debe partir desde un depósito central y retornar a él, respetando siempre la restricción de capacidad [3].

- **Ventanas de tiempo (VRPTW)**: Donde cada cliente debe ser atendido dentro de un intervalo de tiempo específico. Las ventanas de tiempo añaden complejidad porque no solo se optimiza la distancia de las rutas, sino también el tiempo, considerando que los vehículos pueden llegar temprano (y esperar con costo de tiempo de espera) o llegar tarde (incurriendo en penalizaciones). Esta restricción introduce la dimensión temporal explícita al problema de enrutamiento [3].

- **Múltiples depósitos (MDVRP)**: Donde los vehículos pueden iniciar y terminar sus rutas desde diferentes ubicaciones, en lugar de un único depósito central. Esto introduce complejidad adicional porque el algoritmo debe decidir no solo las rutas de visita, sino también asignar qué vehículos operan desde qué depósitos. El MDVRP es relevante en contextos logísticos donde múltiples centros de distribución sirven diferentes regiones geográficas [3].

Más allá de estas extensiones fundamentales, existen numerosas variantes del VRP derivadas de la diversidad de reglas de operación [4]. La revisión de Cáceres-Cruz et al. [3] documenta, entre otras, problemas con *pickup* y *delivery* (recogida y entrega en diferentes ubicaciones) y problemas con entregas divididas (*split delivery*: la demanda de un cliente se reparte entre

varios vehículos). Cada una de estas extensiones añade dimensiones de realismo operacional, pero también incrementa considerablemente la dificultad de encontrar soluciones óptimas.

La progresión desde el VRP clásico hacia variantes más complejas establece el contexto para entender el Problema de Enrutamiento de Entrega de Comida (MDRP). El MDRP no es simplemente una combinación de estas restricciones estáticas; es un problema fundamentalmente dinámico en el que nuevas órdenes arriban continuamente durante el día operativo, requiriendo que las decisiones de enrutamiento se reoptimicen periódicamente. Este cambio de paradigma de problemas estáticos a dinámicos, junto con la necesidad de considerar redes viales reales, sitúa la motivación y relevancia del presente trabajo.

1. 1.3. Última milla y contexto urbano

La última milla de entrega—el segmento final desde un centro de distribución o punto de recogida hasta el domicilio del cliente—es uno de los segmentos más exigentes de la logística urbana. Jazemi et al. [5] documentan que la entrega a zonas residenciales urbanas se complica por el volumen de entregas, los horarios de entrega estrechos y las condiciones de entrega en cambio continuo, y revisan las estrategias de ruteo propuestas para enfrentarlo.

En el contexto de entrega de comida rápida, la última milla presenta desafíos específicos adicionales: (1) alta urgencia—los clientes esperan entregas en ventanas de 30-60 minutos desde la orden, significativamente más restrictivas que en logística general; (2) perecedibilidad—la comida debe mantener temperatura, frescura y presentación, implicando rutas directas sin desviaciones innecesarias; (3) demanda dinámica e impredecible—los flujos de órdenes no son uniformes sino que presentan picos pronunciados (horas de almuerzo, cena), requiriendo reoptimización continua de asignaciones.

Estas características transforman la última milla de comida en un problema distinto del VRP clásico: es inherentemente dinámico, con ventanas de tiempo estrictas y demanda estocástica. Aunque la literatura de enrutamiento dinámico [6, 7] y las estrategias de anticipación [8] han abordado componentes de este desafío, la entrega de comida presenta particularidades únicas que requieren una integración específica de estos conceptos, como se explora en la siguiente sección.

1. 1.4. Enrutamiento dinámico: DVRP y DARP

Mientras que el VRP clásico y sus extensiones (CVRP, VRPTW, MDVRP) asumen que todas las demandas de clientes son conocidas *ex-ante* (antes de la ejecución), la realidad operacional de los sistemas de entrega urbana es fundamentalmente distinta: nuevas órdenes llegan continuamente durante el día, y las decisiones de despacho deben reoptimizarse en tiempo real. Este cambio de paradigma de lo estático a lo dinámico define una nueva clase de problemas de enrutamiento que requieren políticas de despacho adaptativas.

1. 1.5. DVRP: Definición y características fundamentales

En su tesis doctoral, Larsen [9] define el Problema de Enrutamiento de Vehículos Dinámico (DVRP) y propone medidas de su grado de dinamismo. Lo que distingue a un DVRP de un VRP estático es el momento en que la información se da a conocer: parte de ella, en particular las nuevas solicitudes de clientes, se revela durante la operación, por lo que el conjunto de clientes a atender no se conoce por completo al inicio. En términos generales, el DVRP presenta las siguientes características:

- **Llegada estocástica de solicitudes**: Las órdenes no existe como conjunto fijo, sino que arriban según un proceso estocástico.
- **Reoptimización periódica**: El plan de enrutamiento no es estático sino que debe recalcularse continuamente. Las decisiones previas (repartidores ya en ruta) no pueden revertirse completamente, pero las nuevas decisiones pueden considerar órdenes recién llegadas.
- **Incertidumbre temporal**: No solo la demanda es incierta, sino también los tiempos de viaje pueden variar por congestión u otros factores, lo que crea desviaciones entre el plan y la ejecución.
- **Disyuntiva decisión-información**: Cada política de despacho enfrenta un dilema: decidir inmediatamente (riesgo de decisiones subóptimas) vs. esperar más información (riesgo de incumplir criterios de servicio). Pillac et al. [10] profundizan en este aspecto, clasificando los problemas según su grado de dinamismo y la calidad de la información disponible.

Esta caracterización es central para entender la última milla de entrega: cada vez que un cliente coloca una orden en una plataforma de entrega de comida (Grubhub, Uber Eats, DoorDash), el sistema de despacho enfrenta una decisión urgente: asignar esa orden a un repartidor para que sea recogida y entregada. La decisión no puede posponerse indefinidamente (los clientes esperan entregas en 30-60 minutos), pero tampoco puede hacerse de forma completamente reactiva sin consideración del contexto futuro.

1. 1.6. DARP: Del transporte de pasajeros a la entrega de bienes

Un problema históricamente relacionado pero conceptualmente distinto es el Dial-a-Ride Problem (DARP). En [6], Psaraftis formuló el primer modelo dinámico de *dial-a-ride*: en sistemas de transporte compartido (típicamente para pasajeros), usuarios solicitan recogida en una ubicación y entrega en otra ubicación diferente, y ambas solicitudes llegan en tiempo real. El trabajo resolvió mediante programación dinámica la secuenciación óptima de recogidas y entregas de un solo vehículo, ante solicitudes de servicio inmediato.

Cordeau y Laporte [7] revisan la literatura del *Dial-a-Ride Problem*: describen sus características principales y resumen los modelos y algoritmos más importantes, en sus versiones estática y dinámica. Entre las restricciones estructurales del DARP que describen se encuentran:

- **Restricción de capacidad**: Cada vehículo tiene límite de pasajeros simultáneos.
- **Restricción de secuencia temporal**: Un pasajero debe ser recogido antes de ser entregado, con límite temporal máximo entre recogida y entrega.
- **Ventanas de tiempo**: Tanto recogida como entrega pueden tener ventanas horarias de disponibilidad.
- **Criterios de optimización**: Minimización de distancia total o tiempo total de espera, frecuentemente con penalizaciones por incumplimiento de compromisos de tiempo.

Aunque el DARP fue originalmente formulado para transporte de pasajeros, su estructura conceptual es directamente transferible a problemas de entrega de bienes. De hecho, la estructura matemática es análoga: en lugar de "recoger un pasajero en ubicación A y entregar en ubicación

B", se tiene "recoger una orden en restaurante A y entregar en domicilio B". Sin embargo, existen diferencias operacionales críticas:

- En transporte de pasajeros, la restricción de secuencia es rígida (recogida antes de entrega), mientras que en entrega de comida existe más flexibilidad: múltiples órdenes de diferentes restaurantes pueden incluirse en una sola ruta.
- En transporte de pasajeros, el tiempo entre recogida y entrega es determinístico (depende de la ruta), mientras que en comida existe heterogeneidad: tiempo de preparación varía por restaurante, impactando cuando la orden está disponible para recogida.
- En transporte de pasajeros, la experiencia del cliente es inmediata (espera observable), mientras que en comida, el cliente está dispuesto a esperar mientras el pedido se prepara, reduciendo la presión sobre latencia de decisión de despacho.

1. 1.7. El valor de la información futura y el enfoque de Horizonte Rodante

A pesar de sus diferencias, DVRP y DARP comparten la disyuntiva fundamental entre decisiones reactivas (FCFS) y decisiones anticipatorias.

Ichoua et al. [8] abordaron directamente esta disyuntiva en el despacho de vehículos en tiempo real. Compararon una política que desconoce las solicitudes futuras (puramente reactiva) contra una estrategia que incorpora conocimiento probabilístico de las llegadas futuras, representado mediante clientes ficticios en las rutas. En sus experimentos, la estrategia anticipatoria redujo el número de clientes no servidos entre 3.4 % y 29.0 % y la suma de tiempo de viaje y retrasos entre 2.0 % y 16.0 % (9.0 % y 4.2 % en promedio), lo que muestra que la anticipación moderada aporta ganancias sin requerir información perfecta sobre el futuro.

Esta idea fundamental conecta DVRP/DARP con el concepto de horizonte rodante (*rolling horizon*): en lugar de una asignación inmediata (FCFS), el sistema puede esperar un breve período (el horizonte Δ, típicamente 10-30 minutos), acumular un lote de solicitudes cuyo arribo es conocido con certeza relativa, optimizar globalmente ese lote resolviendo un VRP estático (el *matching* óptimo), y luego ejecutar la primera etapa de esa solución optimizada. En la siguiente época, repite el proceso.

1. 1.8. MDRP y políticas de despacho

El Problema de Enrutamiento de Entrega de Comida (MDRP) fue formalmente introducido por Reyes et al. [13] como una nueva clase de problema de enrutamiento dinámico que, aunque comparte características fundamentales con DVRP y DARP, añade dimensiones específicas de la entrega de alimentos que lo distinguen conceptualmente y requieren políticas de despacho especializadas.

1. 1.9. Características compartidas con DVRP y DARP

Como todo problema DVRP, el MDRP hereda la característica de dinamicidad: nuevas órdenes arriban continuamente en tiempo real, y el conjunto de demanda no es conocido *ex-ante*. Como problema similar a DARP, el MDRP hereda la estructura de recogida-entrega: una orden debe ser recogida en un restaurante (fuente) y entregada en un domicilio (destino), con restricciones de ventanas de tiempo y capacidad de vehículos. Estas características mencionadas crean el contexto operacional básico.

Sin embargo, Reyes et al. [13] argumentan que estas dos extensiones no capturan plenamente la realidad de la entrega de comida en ultima milla. El MDRP fue definido para llenar esta brecha, formulando un problema que integra cuatro características estructurales que no aparecen de forma acoplada en la literatura previa:

1. 1.10. Definición y características del MDRP

Reyes et al. [13] definen el MDRP como una clase de problema de recolección y entrega dinámico (DPDP) caracterizado por una *urgencia extrema* y menores oportunidades de consolidación en comparación con la logística tradicional. Sus características distintivas son:

1. **Múltiples puntos de recolección (Restaurantes):** A diferencia del VRP clásico con un depósito central, el MDRP opera con múltiples restaurantes dispersos, cada uno actuando como un punto de recolección independiente con ubicación fija.

2. **Tiempos de preparación:** Las órdenes no están listas inmediatamente al ser colocadas. Reyes et al. [13] introducen el concepto de *ready time* (tiempo de preparación) como una

restricción dura: el courier no puede recoger la orden antes de que la comida esté lista. Esto crea una holgura de tiempo natural pero reduce la flexibilidad del enrutamiento.

3. **Alta urgencia y Dinamismo:** La mayoría de las solicitudes se revelan durante la operación (dinamismo) y deben completarse en ventanas de tiempo muy cortas, típicamente menos de una hora (urgencia). Reyes et al. [13] señalan que esta combinación de dinamismo y urgencia es el desafío central del MDRP.

4. **Agrupamiento dinámico (*bundling*):** Para mejorar la eficiencia, el MDRP permite combinar múltiples órdenes de un mismo restaurante en una sola ruta, creando agrupamientos (referidos en la literatura inglesa como *bundles*). Aunque el modelo teórico de Reyes et al. [13] asume capacidad ilimitada para estos agrupamientos, en la implementación práctica de esta tesis se considera la capacidad física limitada de los vehículos (motocicletas/bicicletas) como una restricción operativa real.

1. 1.11. Enrutamiento realista en redes viales

La validación efectiva de políticas de despacho dinámico depende críticamente de la calidad de los datos de enrutamiento utilizados en la simulación. La literatura actual presenta una división instrumental: por un lado, las plataformas comerciales (Google Maps, Mapbox) ofrecen tiempos de viaje precisos, pero el costo del software con licencia limita su uso en investigación [11] y su naturaleza de *caja negra* impide auditar los algoritmos y reproducir exactamente los experimentos. Por otro lado, la investigación académica tradicional frecuentemente opta por simplificaciones euclidianas que, aunque gratuitas y totalmente reproducibles, carecen de fidelidad operativa.

Las distancias por la red vial son sistemáticamente mayores que las distancias en línea recta: el factor de desviación entre ambas suele ubicarse entre 1.2 y 1.4 en la mayoría de las ciudades (Cole y King, citado por Boyacı et al. [12]). Boyacı et al. [12] evaluaron, con 96 instancias construidas sobre la red vial de 12 ciudades, qué tan buena es la aproximación euclidiana para resolver problemas de ruteo. Concluyeron que, en problemas sin restricciones temporales, la aproximación da en general resultados aceptables, pero advirtieron que en problemas con ventanas de tiempo, sobre todo estrechas, una ruta factible con distancias euclidianas puede volverse infactible al recorrerse por la red real. La entrega de comida pertenece precisamente a

esta última clase. En este sentido, Jazemi et al. [5] señalan que la entrega en zonas residenciales urbanas enfrenta grandes volúmenes, horarios de entrega estrechos y condiciones cambiantes, lo que exige estimaciones de tiempo de arribo (ETA) confiables.

1. 2. Planteamiento del problema

1. 2.1. El problema de decisión: coordinar o reaccionar

Una plataforma de entrega de comida debe decidir, a lo largo del día y sin conocer los pedidos futuros, qué repartidor recoge cada orden y cuándo lo hace: es el problema MDRP descrito en la Sección 1.1. Existen dos maneras básicas de tomar esa decisión. La primera es **reactiva**: cada orden se asigna en cuanto llega al repartidor libre más cercano (*First-Come, First-Served*, FCFS); es simple, inmediata y cercana a la forma en que se despacha manualmente. La segunda es **coordinada**: el sistema espera unos minutos, agrupa órdenes del mismo restaurante y resuelve de forma conjunta qué repartidor atiende cada agrupamiento, como en la heurística de horizonte rodante (*Rolling Horizon*, RH) de Reyes et al. [13].

Ninguna de las dos es mejor en todas las condiciones. Coordinar tiene un costo —algunas órdenes esperan a que se forme el agrupamiento o a que se entregue otra orden antes que la suya— y un beneficio —cada viaje atiende más pedidos—. El balance entre ambos depende, sobre todo, de cuántos repartidores hay disponibles frente a la demanda: con una flota holgada, la espera puede no compensar; con una flota escasa, el ahorro de viajes puede ser decisivo. Para un operador, la pregunta práctica es, por tanto, *en qué condiciones conviene coordinar los despachos*.

1. 2.2. La brecha: ciudades medias y redes viales reales

La evidencia disponible para responder esa pregunta proviene de entornos distintos al de una ciudad media. Reyes et al. [13] evaluaron RH con instancias que reproducen áreas metropolitanas a partir de datos históricos de una plataforma estadounidense, y calcularon los tiempos de viaje como la distancia euclidiana multiplicada por un factor constante. En instancias pequeñas resueltas a optimalidad por Yıldız and Savelsbergh [14], su heurística obtiene un *Click-to-Door* promedio apenas un 4 % superior al óptimo, pero no reportan una comparación directa contra

una política reactiva como FCFS ni analizan cómo cambia la ventaja de coordinar con el tamaño de la flota.

A ello se suma la forma de medir los tiempos de viaje. Las distancias euclidianas subestiman los trayectos reales en un factor típico de 1.2 a 1.4, y Boyacı et al. [12] advierten que, en problemas con ventanas de tiempo estrechas como la entrega de comida, esa subestimación puede producir rutas infactibles y estimaciones de entrega poco confiables. Una evaluación que ignore la red vial puede, entonces, equivocarse no solo en los tiempos de entrega, sino también en la carga real de la flota y, con ella, en el beneficio de coordinar. Las herramientas comerciales de enrutamiento resuelven la precisión, pero tienen costos de licenciamiento [11] y funcionan como cajas negras que dificultan la reproducibilidad; las herramientas de código abierto, como OSRM sobre OpenStreetMap, permiten hacer esta evaluación de forma local, auditable y sin costo de licencias.

1. 2.3. El caso de La Paz, B.C.S.

La Paz es una ciudad media donde no se dispone de datos públicos de operaciones de entrega de comida. Su red vial no es un plano ideal: la línea de costa de la ensenada y la dependencia de pocas arterias estructurantes (como el Blvd. Forjadores o Pino Pallas) obligan a recorridos más largos que la línea recta. Cuánto se alargan los trayectos en su zona urbana, y si esa diferencia modifica la conveniencia de coordinar los despachos, son preguntas sin respuesta. Sin esa evidencia, los desarrolladores y las empresas locales carecen de una base para decidir si un sistema de despacho coordinado se justifica frente a la asignación reactiva.

1. 2.4. Preguntas de investigación

De lo anterior se desprende la pregunta central de esta tesis:

> *En una ciudad media como La Paz, B.C.S., ¿en qué condiciones de saturación de la flota la política de despacho coordinada (Rolling Horizon) supera a la asignación reactiva (FCFS) en tiempo de entrega y nivel de servicio, y cuánto cambia esa evaluación al usar la red vial real en lugar de distancias euclidianas?*

La pregunta se desglosa en tres preguntas específicas:

PI1. ¿Cómo varía la ventaja de RH sobre FCFS con la saturación de la flota, y a partir de qué nivel de saturación conviene coordinar los despachos?

PI2. ¿Cuánto más largos son los trayectos por la red vial de La Paz que en línea recta, y cuánto cambia la comparación entre políticas al usar la red real en lugar de distancias euclidianas?

PI3. ¿Puede realizarse esta evaluación de forma reproducible, con recursos de cómputo modestos y exclusivamente con herramientas de código abierto?

Para responderlas se construye un entorno de simulación sobre la red vial de La Paz (objetivo general y objetivos específicos 1 a 3, Sección 1.3), que incluye un control con distancias euclidianas (objetivo específico 2), y se cuantifican estadísticamente las diferencias entre ambas políticas en cuatro niveles de saturación de la flota (objetivo específico 4). Las respuestas se presentan en el Capítulo 4 y se sintetizan en la Sección 5. 1.

1. 3. Objetivos

1. 3.1. Objetivo general

Evaluar la eficiencia operativa de la política de despacho *Rolling Horizon* frente a una estrategia reactiva (FCFS) en el problema MDRP, mediante el desarrollo de un entorno de simulación reproducible que integre restricciones topológicas reales de la ciudad de La Paz, B.C.S. y patrones de demanda estocásticos.

1. 3.2. Objetivos específicos

- Generar un conjunto de instancias sintéticas calibradas que proyecten los patrones temporales de demanda de la literatura (LaDe) sobre el área urbana de La Paz, garantizando la validez geoespacial de los puntos de entrega y recolección.
- Integrar un motor de simulación de eventos discretos con la infraestructura de enrutamiento OSRM sobre cartografía OpenStreetMap, validando la conectividad del grafo vial y la precisión de los tiempos de viaje frente a distancias euclidianas.

- Implementar computacionalmente la heurística de Horizonte Rodante (*Rolling Horizon*) con capacidad de agrupamiento dinámico (*bundling*) y la política de control *First-Come, First-Served*, instrumentando la recolección de métricas de desempeño.
- Cuantificar las diferencias de desempeño entre ambas políticas mediante análisis estadístico de los KPIs críticos: tiempos *Click-to-Door*, latencia *Ready-to-Pickup* y eficiencia de la flota, determinando la significancia de las mejoras observadas.

1. 4. Hipótesis

La implementación de una política de despacho anticipatoria (Horizonte Rodante), alimentada con una matriz de costos de viaje reales (OSRM) sobre la topología de La Paz, superará significativamente en eficiencia operativa "órdenes por hora" y nivel de servicio "*Ready-to-Door*" a la estrategia reactiva estándar (FCFS), demostrando la viabilidad de soluciones de optimización logística de alto rendimiento basadas exclusivamente en software de código abierto.

1. 5. Justificación

La creciente demanda de servicios de entrega bajo demanda *on-demand delivery* ha transformado la logística urbana, imponiendo desafíos que los métodos exactos tradicionales no pueden resolver en tiempo real debido a su complejidad exponencial. Si bien el modelo de *Rolling Horizon* [13] ofrece una solución teórica prometedora, su aplicabilidad práctica en ciudades latinoamericanas de escala media se ve limitada por la carencia de validación en topologías viales restrictivas y la dependencia de costos prohibitivos de software propietario. La presente investigación se justifica bajo las siguientes dimensiones:

1. 5.1. Relevancia Científica

Existe una disonancia documentada entre los modelos teóricos de enrutamiento, que asumen espacios euclidianos o redes ideales, y la realidad operativa de la última milla. Figliozzi [15] muestra que los kilómetros que generan los recorridos urbanos dependen fuertemente de las restricciones de ruteo (número de paradas, ventanas de tiempo) y de la distancia entre el centro

de distribución y el área de servicio, factores que los modelos simplificados no capturan. El presente trabajo contribuye a cerrar esa brecha en el contexto de la entrega de comida: aporta evidencia sobre las condiciones de saturación en las que la heurística de horizonte rodante supera a la asignación reactiva en una ciudad media, y mide cuánto cambia esa comparación al sustituir las distancias euclidianas por la red vial real de La Paz, B.C.S.

1. 5.2. Aporte Tecnológico y Ciencia Abierta

Este proyecto contribuye con una arquitectura de software diseñada bajo principios de Ciencia Abierta. Se integra el motor de enrutamiento OSRM desplegando su imagen oficial en Docker, lo que permite sustituir las distancias euclidianas por tiempos de viaje reales sobre la red vial, asegurando la consistencia del entorno de cálculo [16]. Esta integración desacopla la complejidad de la infraestructura de enrutamiento de la lógica de simulación, facilitando que la comunidad académica replique los experimentos sin enfrentar problemas de configuración.

1. 5.3. Impacto Económico y Social

Para las PyMES y startups locales en Baja California Sur, la dependencia de APIs comerciales (como Google Maps) representa una barrera de entrada económica. Al demostrar la viabilidad técnica de optimizar rutas utilizando exclusivamente herramientas *Open Source* (OSRM), esta tesis ofrece una alternativa de *soberanía tecnológica* que puede reducir los costos de licenciamiento. Esto podría facilitar el acceso a herramientas de logística avanzadas a empresas locales que hoy compiten con plataformas transnacionales.

1. 5.4. Delimitación frente a métodos de Aprendizaje Profundo

Aunque existen métodos avanzados de Inteligencia Artificial (como Aprendizaje Profundo), estos requieren grandes cantidades de datos históricos o de simulación para entrenarse [17]. En ciudades como La Paz, donde no se dispone de registros accesibles de rutas y pedidos, estos modelos enfrentan el problema de "arranque en frío": no pueden operar sin entrenamiento previo.

Por ello, esta tesis utiliza *Rolling Horizon*, un algoritmo que no necesita datos históricos y funciona desde el primer día de operación. Además, al ser computacionalmente ligero (Sección 4.

8), puede ejecutarse en equipos de cómputo convencionales. Un sistema de este tipo en operación podría, a su vez, generar los datos necesarios para entrenar modelos de IA en el futuro.

1. 6. Alcance y limitaciones

El presente trabajo de investigación se enfoca en el desarrollo, implementación y evaluación de un entorno de simulación para el problema MDRP. A continuación se detallan las fronteras funcionales, geográficas y técnicas del proyecto, así como las restricciones operativas asumidas.

1. 6.1. Alcance Geográfico y de Datos

Infraestructura Vial (La Paz, B.C.S.)

Geográficamente nos limitamos estrictamente a la zona urbana de La Paz, Baja California Sur. Se implementa una instancia local del servidor de enrutamiento OSRM v5.27.1 que opera sobre `localhost`, eliminando la dependencia de conectividad externa a internet y garantizando latencia mínima. Este servidor procesa el extracto en formato `.pbf` de OpenStreetMap correspondiente a la ciudad de La Paz.

Instancias de Demanda y Patrones Temporales

El alcance se limita a la generación de **instancias sintéticas calibradas**, construidas bajo los siguientes parámetros:

- **Volumen:** Escenarios de aproximadamente 1,000 órdenes distribuidas en una ventana operativa de 11 horas (08:00 - 19:00).
- **Patrones Estocásticos:** Distribución de pedidos con dos picos diarios, tomados del perfil de recolección de paquetes del dataset de referencia *LaDe* [18] (9:00 y 17:00); no corresponden a los horarios de comida locales.

1. 6.2. Alcance Funcional y Algorítmico

El sistema compara exclusivamente dos políticas de despacho::

1. **Política Base *First-Come-First-Served* (FCFS):** Asignación reactiva inmediata, en el momento en que se coloca cada orden, al repartidor libre más cercano al restaurante, sin consolidación de órdenes.

2. **Política Propuesta (*Rolling Horizon*):** Implementación de la heurística de Reyes et al. [13] con un horizonte de asignación configurable (20 minutos en los experimentos) y capacidad de agrupamiento (*bundling*) dinámico de hasta 4 órdenes por repartidor.

3. **Escenario de Control (Aislamiento de Variables):** Ambas políticas se ejecutan además con distancias en línea recta (Haversine) recorridas a la velocidad media de la red OSRM, de modo que la única diferencia con el escenario principal es la geometría de la red vial. Esto permite separar la contribución del algoritmo de optimización de la atribuible a la topología real del mapa.

La simulación se ejecuta bajo un esquema de **pasos de tiempo fijos**, una simulación donde el estado del sistema se evalúa y actualiza en intervalos regulares de reoptimización ($f = 5$ min). Este enfoque se alinea con la naturaleza periódica del algoritmo *Rolling Horizon* propuesto por Reyes et al. [13], permitiendo capturar la estocasticidad en la llegada de órdenes mientras se mantiene la sincronización con los ciclos de decisión de la flota.

1. 6.3. Alcance Técnico: Reproducibilidad e Infraestructura

El proyecto entrega software reproducible, diseñado para facilitar la auditoría y la extensión futura. El alcance técnico garantiza:

- **Infraestructura de Enrutamiento Contenerizada:** El motor de enrutamiento OSRM se despliega mediante **Docker** (imagen `ghcr.io/project-osrm/osrm-backend:v5.27.1`), configurado con el algoritmo CH (*Contraction Hierarchies*). Esto asegura que el entorno de cálculo de rutas sea idéntico en cualquier máquina anfitriona, eliminando discrepancias por versiones de librerías del sistema operativo.

- **Independencia de APIs:** El sistema opera de forma autónoma (*offline*) sobre `localhost`, sin consumo de créditos de Google Maps o Mapbox, eliminando barreras económicas para la replicación de experimentos.

- **Requisitos Computacionales:** El entorno está diseñado para ejecutarse en hardware de consumo estándar (CPU x86/ARM, 8GB RAM), democratizando el acceso a herramientas de optimización logística avanzada.

1. 6.4. Supuestos y Limitaciones

Las conclusiones del estudio deben interpretarse bajo las siguientes restricciones metodológicas y simplificaciones del modelo:

Modelo de Tráfico Estático

El cálculo de tiempos de viaje asume velocidades promedio constantes por tipo de vía (perfil `car.lua` ajustado). **Limitación:** No se modela la congestión vehicular dinámica en tiempo real (accidentes o cierres viales temporales). Esto implica que las métricas de tiempo de entrega (*Click-to-Door*) representan un caso optimista respecto a la realidad operativa con tráfico saturado.

Simplificación de la Logística de Última Milla

El modelo asume tiempos de servicio deterministas divididos en dos componentes, aplicados con la convención de Reyes et al. [13] (la mitad del tiempo antes y la mitad después del instante de recolección o de entrega):

- **Tiempo de Recolección (*Pickup*):** Se aplica una penalización fija de $T_{pickup} = 4$ min por visita a cada restaurante asignado por visitar. Si la comida aún no está lista, el repartidor espera en el restaurante.
- **Tiempo de Entrega (*Drop-off*):** Se añade un tiempo de servicio de $T_{drop} = 4$ min por cada orden entregada al cliente.
- **Ausencia de Variabilidad Estocástica:** No se modelan retrasos aleatorios por búsqueda de estacionamiento, acceso a edificios complejos o clientes ausentes; los tiempos son constantes.

Comportamiento de la Flota

- **Velocidad Constante:** Los repartidores viajan a la velocidad constante estimada por OSRM, sin desviaciones por comportamiento humano.
- **Sin Retorno a Base:** Al finalizar una entrega, los repartidores permanecen estáticos en la ubicación del último cliente hasta recibir una nueva asignación. No se modelan estrategias de reposicionamiento o retorno a zonas de alta demanda.
- **Energía y combustible** No se modelan paradas técnicas por recarga de combustible o batería durante el turno operativo.

Limitaciones Algorítmicas

- **Heurística de Asignación Jerárquica:** Siguiendo la metodología de Reyes et al. [13], se adopta una estrategia secuencial que prioriza órdenes críticas (Grupos de prioridad).
- **Compromiso en Dos Etapas:** La lógica de *compromiso parcial* permite redirigir a un repartidor en ruta, lo cual es una simplificación operativa que asume obediencia total y comunicación instantánea sin fricción.

Escenario Nominal de Operación

La simulación asume condiciones de flujo ideal. Quedan fuera del alcance:

- Cancelaciones de órdenes por parte de clientes o restaurantes.
- Rechazo de asignaciones por parte de los repartidores.
- Fallas mecánicas de la flota o gestión de efectivo.
- *Multihoming* (repartidores trabajando para múltiples apps simultáneamente).
- **Homogeneidad de carga:** Se asume que todas las órdenes poseen dimensiones estándar compatibles con la mochila del repartidor, ignorando restricciones de volumen físico o peso excesivo.

Generalización de Patrones de Consumo

La forma diaria de la demanda se toma del dataset LaDe [18] y se asume transferible al contexto de La Paz. Este supuesto no está respaldado por datos locales; el tipo de ciudad y su infraestructura influyen en el comportamiento del transporte [19], por lo que la transferibilidad del patrón es una limitación del estudio.

Capítulo 2

Marco teórico

Este capítulo presenta los fundamentos teóricos que sustentan el diseño del simulador y las políticas de despacho implementadas en el Capítulo 3. Se formaliza el Meal Delivery Routing Problem (MDRP), se describe la heurística de horizonte rodante, se revisan las técnicas de enrutamiento sobre grafos viales y se introduce el modelo estocástico de generación de demanda.

2. 1. Formulación matemática del MDRP

El **Meal Delivery Routing Problem** (MDRP), introducido por Reyes et al. [13] y estudiado también por Yıldız and Savelsbergh [14], con un modelo de información perfecta, y por Ulmer et al. [20], bajo el rótulo de *Restaurant Meal Delivery Problem*, formaliza la clase de problemas de enrutamiento dinámico que surgen en las operaciones de entrega de comida bajo demanda. Esta sección reproduce la notación y las suposiciones estructurales del modelo de Reyes et al. [13], que constituyen la base formal del simulador desarrollado en este trabajo. Las formulaciones relacionadas difieren en aspectos que este trabajo no adopta: Yıldız and Savelsbergh [14] obtienen soluciones con cotas de calidad demostrables suponiendo que todas las órdenes se conocen de antemano, y Ulmer et al. [20] tratan el *ready time* como incierto. Aquí, como en Reyes et al. [13], el *ready time* se genera aleatoriamente pero el despachador lo conoce desde que se coloca la orden.

2. 1.1. Conjuntos y entidades

El MDRP se define sobre tres conjuntos fundamentales:

- $\mathcal{R}$: conjunto de **restaurantes**. Cada restaurante $r \in \mathcal{R}$ posee una ubicación fija ℓ_r.
- $\mathcal{O}$: conjunto de **órdenes** (pedidos). Cada orden $o \in \mathcal{O}$ se caracteriza por:
  - restaurante de origen $r_o \in \mathcal{R}$,
  - instante de colocación (*placement time*) a_o,
  - instante de disponibilidad (*ready time*) $e_o \geq a_o$,
  - ubicación de entrega (*drop-off*) ℓ_o.

  La información de una orden o se revela al sistema únicamente en el instante a_o (llegada dinámica).
- $\mathcal{C}$: conjunto de **repartidores** (couriers). Cada courier $c \in \mathcal{C}$ posee:
  - instante de inicio de turno (*on-time*) e_c,
  - ubicación inicial ℓ_c en el instante e_c,
  - instante de fin de turno (*off-time*) $l_c > e_c$.

  A diferencia de las órdenes, toda la información sobre restaurantes y couriers se conoce *a priori*.

El MDRP consiste en determinar rutas factibles para los couriers que completen la recolección y entrega de las órdenes, optimizando una o varias métricas de desempeño.

2. 1.2. Función objetivo

Sea x una asignación factible de órdenes a couriers que cumple las restricciones de capacidad, de secuenciamiento (recolección antes de entrega) y de ventanas temporales del MDRP. Reyes et al. [13] no fijan una función objetivo única: plantean el problema con varias métricas de desempeño y ajustan su algoritmo con un objetivo jerárquico (primero entregar una proporción muy alta de las órdenes y después minimizar el *Click-to-Door* promedio). Con fines expositivos, esas prioridades pueden resumirse como una combinación ponderada de tiempo total de servicio al cliente, compensación de la flota y órdenes no atendidas:

$$\min_{x} \ \alpha_1 \sum_{o \in \mathcal{O}} \mathrm{CtD}_o(x) \ + \ \alpha_2 \sum_{c \in \mathcal{C}} \mathrm{Pago}(c; x) \ + \ \alpha_3 \, |\mathcal{O}_u(x)| \tag{2.1}$$

donde $\mathrm{CtD}_o(x)$ es el tiempo *Click-to-Door* inducido por la asignación x, $\mathrm{Pago}(c; x)$ es la compensación del courier c bajo el modelo dual descrito más adelante, $\mathcal{O}_u(x)$ es el conjunto de órdenes no entregadas al cierre del horizonte de simulación, y $\alpha_1, \alpha_2, \alpha_3 \geq 0$ son pesos que reflejan las prioridades de la plataforma [13, 20]. En la heurística de horizonte rodante (Sección 2.2) el objetivo (2.1) no se construye explícitamente: se aproxima miópicamente en cada época mediante el peso $w_{s,d}$ del modelo de asignación bipartita, y la evaluación *a posteriori* se realiza desagregando las tres componentes en los indicadores descritos en la Sección 2.4 y el Capítulo 4.

2. 1.3. Suposiciones estructurales

Las siguientes suposiciones, tomadas directamente de Reyes et al. [13], definen la factibilidad de las soluciones:

1. **Agrupamiento (*bundling*):** Órdenes del mismo restaurante pueden combinarse en un agrupamiento (*bundle*) con múltiples puntos de entrega. El *ready time* del agrupamiento es el máximo *ready time* de sus órdenes constituyentes.

2. **Tiempos de viaje:** El tiempo de viaje entre cualquier par de ubicaciones se asume invariante en el tiempo.

3. **Tiempos de servicio:** Se definen dos tiempos de servicio constantes: s^r para la recolección en un restaurante (independiente del número de órdenes) y s^o para la entrega al cliente.

4. **Restricción de turno:** Un courier no puede iniciar nuevas recolecciones después de su *off-time* l_c, pero puede completar entregas pendientes.

5. **Obediencia determinista:** En el marco determinista del modelo, los couriers aceptan todas las instrucciones y esperan nuevas asignaciones en su última ubicación conocida.

2. 1.4. Modelo de compensación

La remuneración de los couriers sigue un esquema de **compensación mínima garantizada** [13]: cada courier percibe el máximo entre su ganancia por entregas y un pago mínimo por hora:

$$\text{Pago}(c) = \text{máx}(p_1 \cdot n_c,\ \ p_2 \cdot (l_c - e_c)) \tag{2.2}$$

donde n_c es el número total de órdenes entregadas por el courier c, p_1 es el pago por orden y p_2 el pago mínimo por hora. Este modelo alinea los incentivos del courier con los objetivos de la plataforma: cuando la demanda es suficiente, la ganancia variable domina; cuando la demanda es baja, la compensación mínima garantiza un ingreso base.

2. 1.5. Métricas de desempeño

Reyes et al. [13] definen un conjunto de métricas primarias que capturan las perspectivas de los tres actores del sistema (clientes, restaurantes y plataforma). Las más relevantes para este trabajo son:

1. ***Click-to-Door* (CtD):** tiempo entre la colocación y la entrega de la orden, $\text{CtD}_o = t_{delivery,o} - a_o$. Se define un objetivo τ y un máximo absoluto τ_{max}.

2. ***Ready-to-Pickup* (RtP):** tiempo de espera de la comida lista en el restaurante, $\text{RtP}_o = t_{pickup,o} - e_o$.

3. ***Ready-to-Door* (RtD):** tiempo desde la disponibilidad hasta la entrega, $\text{RtD}_o = t_{delivery,o} - e_o$.

4. **Utilización del courier**: fracción del turno dedicada a conducción y servicio (vs. tiempo de espera).

5. **Costo por orden**: compensación total dividida entre el número de órdenes entregadas.

El artículo original define 14 métricas primarias con estadísticas de resumen (media, percentiles 10, 50, 90). La Sección 3.7 detalla los 21 indicadores específicos implementados en este trabajo.

2. 1.6. Variantes del entorno operativo

Reyes et al. [13] identifican dos extensiones del modelo base que amplían las acciones disponibles para el despachador:

- **Preposicionamiento:** Además de asignar rutas de recolección-entrega, el despachador puede enviar a un courier hacia un restaurante anticipadamente (*prepositioning move*).
- **Actualización de asignaciones:** Cuando un courier llega a un restaurante, el agrupamiento asignado puede modificarse (añadir o quitar órdenes) antes de la recolección efectiva.

Ambas extensiones se implementan en el simulador mediante el mecanismo de compromiso en dos etapas descrito en la Sección 3.5: el compromiso parcial (*inbound*) funciona como preposicionamiento, y la reoptimización entre épocas permite la actualización de agrupamientos.

2. 1.7. Clasificación del MDRP en la literatura

El MDRP pertenece a la familia de los problemas dinámicos de recolección y entrega (dPDP), una subclase de los problemas dinámicos de enrutamiento de vehículos (dVRP) [10]. Se distingue de otros dPDP por tres características simultáneas: (1) múltiples puntos de recolección (restaurantes), (2) alta urgencia ($\tau \approx 40$ min) y (3) dinamismo total (todas las órdenes son dinámicas). Estas propiedades descartan los métodos exactos y favorecen heurísticas miopes basadas en horizontes rodantes, como la desarrollada en la Sección 2.2. Steever et al. [21], inspirados en un servicio de entrega de comida del sureste de Estados Unidos, estudian una variante en la que un mismo pedido puede incluir platillos de varios restaurantes (*virtual food court*); en simulación, su heurística que anticipa estados futuros supera a una heurística miope en métricas de servicio al cliente.

2. 2. Heurística de horizonte rodante

La literatura identifica al menos tres familias de políticas anticipatorias para problemas dinámicos de recolección y entrega: la aproximación de la función de valor (*value function approximation*, VFA) mediante simulación offline [22], la asignación anticipatoria de clientes (*anticipatory customer assignment*, ACA) aplicada al RMDP [20], y los esquemas de horizonte rodante [13]. Las dos primeras requieren entrenamiento offline con datos históricos o simulaciones extensivas; la tercera opera con únicamente la información disponible al instante de decisión. Este trabajo adopta el horizonte rodante por su tratabilidad computacional y su independencia

de datos históricos, condición necesaria en un contexto de despliegue inicial (La Paz, B.C.S.) sin trazas operativas previas.

El algoritmo de Reyes et al. [13] resuelve el MDRP mediante un esquema de horizonte rodante (*rolling horizon*) que, cada f minutos, ejecuta un ciclo de agrupamiento, asignación y compromiso sobre el subconjunto de órdenes cuyo *ready time* cae dentro del horizonte de asignación Δ_u. Formalmente, al instante de optimización t el algoritmo opera sobre el conjunto

$$U_t = \{o \in \mathcal{O}_t : e_o \leq t + \Delta_u\},$$

donde $\mathcal{O}_t$ es el conjunto de órdenes reveladas hasta t que aún no han sido finalizadas. Dado que planificar entregas para órdenes cuyo *ready time* es lejano es de escasa utilidad en un entorno altamente dinámico y urgente, restringir la atención a U_t reduce el espacio de decisiones sin sacrificar calidad de servicio.

2. 2.1. Tamaño objetivo de agrupamiento

Para equilibrar la utilización de los couriers con la frescura de las entregas, el algoritmo ajusta dinámicamente el número de órdenes que se agrupan en un mismo viaje. El tamaño objetivo de agrupamiento (*target bundle size*) al instante t se define como la razón entre las órdenes próximas a estar listas y los couriers disponibles:

$$Z_t = \frac{|\{o \in \mathcal{O}_t : e_o \leq t + \Delta_1\}|}{|\{c \in \mathcal{D}_t : e_c^{avail} \leq t + \Delta_2\}|}, \quad \Delta_1 > 0,\ \Delta_2 > 0, \tag{2.3}$$

donde $\mathcal{D}_t$ es el conjunto de couriers en turno al instante t, e_c^{avail} es el momento en que el courier c queda disponible para una nueva asignación, y Δ_1, Δ_2 son horizontes de anticipación calibrados fuera de línea. Conviene distinguir explícitamente los tres horizontes empleados por el algoritmo: Δ_u es el *horizonte de asignación* que delimita qué órdenes entran a la optimización en cada época; Δ_1 es el *lookahead de demanda* usado en el numerador de Z_t para estimar cuántas órdenes estarán pronto listas; y Δ_2 es el *lookahead de oferta* usado en el denominador de Z_t para estimar cuántos couriers estarán pronto disponibles. Para que el algoritmo tenga carácter anticipatorio, Reyes et al. [13] recomiendan $f < \Delta_u \leq \Delta_1$ y $f < \Delta_u \leq \Delta_2$. En la implementación de este trabajo (Sección 3.1, Tabla 3.2) los tres horizontes se fijan en 20 minutos junto con $f = 5$ min, que corresponde a la variante 7 de la malla de parámetros evaluada por Reyes et al. [13]; la igualdad de valores no implica equivalencia conceptual entre los horizontes. Cuando $Z_t < 1$, hay

más couriers que órdenes y el comportamiento converge hacia asignaciones individuales; cuando $Z_t \gg 1$, el sistema está saturado y se favorecen agrupamientos más grandes para maximizar la capacidad de entrega. Si no hay couriers disponibles antes de $t + \Delta_2$, se asigna un valor por defecto.

2. 2.2. Generación de agrupamientos

Una vez determinado Z_t, el algoritmo construye agrupamientos (*bundles*) por restaurante. Para cada restaurante r, se procesan las órdenes de $U_{t,r} = U_t \cap \{o : r_o = r\}$ mediante un procedimiento de inserción paralela con mejora local [13]:

1. **Inicialización:** Se crean $m_r = \max\big(k_t^r,\ \lceil |U_{t,r}|/Z_t \rceil\big)$ agrupamientos vacíos, donde k_t^r es el número de couriers disponibles en el restaurante r. Las órdenes se ordenan por e_o ascendente.[1]

2. **Inserción:** Cada orden se inserta en la posición del agrupamiento que minimiza el costo de inserción:
$$\text{costo}(s, o, p) = \sum_{(i,j)\in s} T(i,j) + \beta \sum_{i \in s} \text{ServiceDelay}(i), \tag{2.4}$$
donde $T(i,j)$ es el tiempo de viaje entre ubicaciones consecutivas de la ruta y $\beta > 0$ pondera la pérdida de frescura. El valor concreto $\beta = 0.1$ adoptado en la implementación (Sección 3.1) está dentro del intervalo $[0, 1]$ que Reyes et al. [13] exploran al ajustar este coeficiente (no publican un valor único recomendado) y mantiene la prioridad del tiempo de viaje como término dominante. Si un agrupamiento alcanza Z_t órdenes, una inserción adicional solo se permite si mejora la eficiencia (reduce el tiempo promedio por orden).

3. **Mejora local (*remove-reinsert*):** Cada orden se extrae de su agrupamiento actual y se reinserta en la posición globalmente óptima entre todos los agrupamientos, corrigiendo asignaciones sub-óptimas del paso anterior.

Cada agrupamiento queda asociado a una ruta abierta: restaurante $\to$ entrega$_1$ $\to$ entrega$_2$ $\to \cdots \to$ entrega$_k$.

[1] La implementación (Sección 3.5) utiliza $m_r = \min(\lceil n/Z_t \rceil, |\mathcal{C}_{disp}|)$, acotando el número de agrupamientos a los couriers disponibles para evitar agrupamientos sin courier posible.

2. 2.3. Esquema de prioridad

Antes de resolver la asignación, los agrupamientos se clasifican en tres grupos de prioridad según su urgencia [13]:

- Grupo i (urgente): el objetivo de *Click-to-Door* τ ya es inalcanzable para al menos una orden del agrupamiento.
- Grupo ii (retrasado): el objetivo τ aún es alcanzable, pero la recolección no puede realizarse en el *ready time* de las órdenes.
- Grupo iii (normal): el agrupamiento puede recogerse y entregarse dentro de los objetivos de servicio.

La asignación se resuelve secuencialmente por grupo (i $\rightarrow$ ii $\rightarrow$ iii), garantizando que los agrupamientos más urgentes obtengan los couriers más cercanos antes de que estos sean consumidos por órdenes menos críticas.

2. 2.4. Modelo de asignación lineal

En cada grupo de prioridad, la asignación de agrupamientos a couriers se formula como un problema de asignación lineal (*linear assignment problem*). Sea S el conjunto de agrupamientos del grupo actual y $\mathcal{D}$ el subconjunto de couriers disponibles. Para cada par $(s, d) \in S \times \mathcal{D}$ se define el peso:

$$w_{s,d} = \frac{N_s}{\text{máx}_{o \in s}\{\delta^s_{o,d}\} - e_d} - \theta\Big(\pi_{s,d} - \max_{o \in s}\{e_o\}\Big), \tag{2.5}$$

donde N_s es el número de órdenes en el agrupamiento s, $\delta^s_{o,d}$ es el instante en que la orden $o \in s$ sería entregada al cliente bajo la ruta inducida al asignar el agrupamiento s al courier d (es decir, el tiempo de finalización de la entrega dentro de la secuencia restaurante $\rightarrow$ entregas), e_d es el momento en que d queda disponible, $\pi_{s,d}$ es el tiempo de recolección del agrupamiento, y $\theta > 0$ penaliza el retraso en la recolección respecto al *ready time*. El primer término captura la eficiencia (órdenes por unidad de tiempo); el segundo, la frescura (demora en el pickup).

El problema se formula como:

$$\max_{x} \quad \sum_{d \in \mathcal{D}} \sum_{s \in S} w_{s,d}\, x_{s,d} \tag{2.6}$$

$$\text{s.a.} \quad \sum_{s \in S} x_{s,d} \leq 1, \quad \forall\, d \in \mathcal{D} \tag{2.7}$$

$$\sum_{d \in \mathcal{D} \cup \{0\}} x_{s,d} = 1, \quad \forall\, s \in S \tag{2.8}$$

$$x_{s,d} \in \{0, 1\} \tag{2.9}$$

La restricción (2.7) garantiza que cada courier reciba a lo más un agrupamiento; la restricción (2.8) asegura que cada agrupamiento sea asignado a exactamente un courier (posiblemente artificial, $d = 0$, para absorber exceso de demanda). Al tratarse de un problema de asignación lineal, puede resolverse de forma exacta en tiempo polinomial mediante el *Algoritmo Húngaro* [23], cuyas implementaciones modernas tienen complejidad $O(n^3)$, lo que garantiza tiempos de ejecución despreciables incluso para instancias con cientos de couriers y agrupamientos.

Reyes et al. [13] exploran también formulaciones de mayor complejidad (partición de agrupamientos por *ready time* y pares de agrupamientos consecutivos), pero el modelo lineal constituye la formulación base del algoritmo y es la adoptada en la implementación de este trabajo (Sección 3.5).

2. 2.5. Estrategia de compromiso en dos etapas

Tras resolver la asignación, no todas las decisiones se comunican de forma inmediata a los couriers. La estrategia de compromiso aditivo en dos etapas (*two-stage additive commitment*) [13] descompone cada asignación tentativa (s, d) en:

1. **Compromiso final:** Si d puede alcanzar el restaurante r_s antes de $t + f$ y todas las órdenes de s estarán listas antes de $t + f$, se instruye a d para recoger y entregar las órdenes de s.

2. **Compromiso parcial (*inbound*):** Si d no puede recoger s antes de $t + f$, pero completa su asignación actual antes de $t + f$, se le instruye únicamente a desplazarse hacia r_s. En la siguiente época, d se reasigna con la garantía de que su agrupamiento incluirá al menos las órdenes de s (propiedad aditiva).

3. **Ignorar:** Si d no puede iniciar una nueva asignación antes de $t+f$, la decisión se pospone.

4. **Excepción de espera prolongada:** Si alguna orden de s lleva lista más de x minutos, se fuerza un compromiso final independientemente de las condiciones anteriores.

El compromiso parcial actúa como *preposicionamiento*: el courier se acerca al restaurante mientras las órdenes terminan de prepararse, y el algoritmo retiene la flexibilidad de actualizar el agrupamiento cuando nueva información se revela en la siguiente época. Esta descomposición es consistente con las extensiones de preposicionamiento y actualización de asignaciones definidas en la Sección 2. 1.6.

La Sección 3.5 describe la implementación concreta de este algoritmo, incluyendo los valores numéricos de los parámetros (f, Δ_u, Δ_1, Δ_2, β, θ, x) y las adaptaciones realizadas para la topología de La Paz.

2. 3. Enrutamiento sobre grafos viales

Las heurísticas descritas en la Sección 2.2 requieren evaluar tiempos de viaje entre pares de ubicaciones en cada época de optimización. Reyes et al. [13] emplean distancias euclidianas multiplicadas por un factor de conversión γ, lo que produce un grafo completo con tiempos de viaje simétricos. Las distancias por la red vial suelen ser entre 1.2 y 1.4 veces las distancias en línea recta (Cole y King, citado por Boyacı et al. [12]), y Boyacı et al. [12] advierten que, en problemas con ventanas de tiempo estrechas, las rutas construidas con distancias euclidianas pueden resultar infactibles sobre la red real. Por su parte, Pereira et al. [11] señalan que los largos tiempos de cómputo y el costo del software con licencia dificultan el enrutamiento en investigación, y ofrecen una alternativa abierta (`r5r`) para calcular tiempos de viaje sobre redes reales. Para eliminar esta fuente de error, el presente trabajo sustituye el modelo euclidiano por consultas de enrutamiento sobre el grafo vial real, empleando la cadena OpenStreetMap → OSRM.

2. 3.1. OpenStreetMap como modelo de datos geográficos

OpenStreetMap (OSM) es un proyecto colaborativo de cartografía abierta que modela la infraestructura vial mediante tres primitivas [24, 25]:

- **Nodos** (`nodes`): puntos georreferenciados `(lat, lon)` que representan intersecciones, semáforos o puntos de interés.
- **Vías** (`ways`): secuencias ordenadas de nodos que representan segmentos de calle. Cada vía porta atributos (`tags`) como tipo de vía (`highway`), límite de velocidad (`maxspeed`), sentido de circulación (`oneway`) y superficie (`surface`).
- **Relaciones** (`relations`): agrupaciones de nodos y vías que representan estructuras complejas (restricciones de giro, rutas de transporte público, etc.).

El grafo vial resultante $G = (V, E, c)$ tiene vértices V correspondientes a los nodos y aristas E derivadas de las vías, con función de costo $c(e)$ definida por la distancia geográfica del segmento y la velocidad permitida. Para la zona urbana de La Paz, B.C.S., el extracto de Geofabrik contiene aproximadamente 10^4 nodos y 10^4 aristas relevantes para el perfil vehicular.

2. 3.2. *Contraction Hierarchies*

Dado un grafo vial $G = (V, E, c)$, el cálculo de caminos mínimos mediante el algoritmo de Dijkstra [26], implementado con montículos de Fibonacci (una mejora posterior al trabajo original), tiene complejidad $O(|E| + |V| \log |V|)$, lo que resulta prohibitivo para consultas frecuentes en grafos con millones de nodos. Geisberger et al. [27] proponen *Contraction Hierarchies* (CH), una técnica de preprocesamiento que permite resolver consultas de camino mínimo en milisegundos. En este trabajo, la ventaja de CH no proviene tanto del tamaño del grafo urbano como del número de consultas: cada época de la política RH evalúa rutas para todos los pares courier–agrupamiento (Sección 2.3.3), y a lo largo de un día simulado se acumulan miles de consultas.

El algoritmo se divide en dos fases:

Preprocesamiento. Los nodos del grafo se ordenan por un criterio de importancia (e.g., diferencia de aristas creadas menos aristas eliminadas) y se contraen secuencialmente. Contraer un nodo v consiste en:

1. Eliminar v y todas sus aristas incidentes del grafo.

2. Para cada par de vecinos (u, w) de v, si el camino mínimo $u \to v \to w$ es el único camino óptimo entre u y w en el grafo residual, insertar una arista atajo (*shortcut*) (u, w) con peso $c(u, v) + c(v, w)$.

El resultado es un grafo aumentado $G^+ = (V, E \cup E_{sc})$ donde E_{sc} es el conjunto de atajos. Cada nodo recibe un nivel jerárquico igual a su orden de contracción.

Consulta. Para encontrar el camino mínimo de s a t, se ejecuta una búsqueda bidireccional en G^+: desde s hacia arriba en la jerarquía (solo aristas hacia nodos de mayor nivel) y desde t hacia arriba. El camino óptimo pasa por el nodo de encuentro que minimiza la suma de distancias parciales. En la red vial de Europa occidental, con unos 18 millones de nodos, las consultas toman menos de 1 ms [27].

2. 3.3. OSRM: motor de enrutamiento

OSRM (*Open Source Routing Machine*) es un motor de enrutamiento de código abierto introducido por Luxen and Vetter [28], que implementa CH sobre datos de OpenStreetMap y expone una API HTTP para consultas en tiempo real. Su cadena de procesamiento consta de tres etapas:

1. `osrm-extract`: parsea el archivo `.pbf` de OSM y genera una representación intermedia del grafo, aplicando un perfil de velocidades por tipo de vía (e.g., `car.lua`).

2. `osrm-contract`: construye la jerarquía CH sobre el grafo extraído, generando archivos de índice que codifican los atajos y la ordenación jerárquica.

3. `osrm-routed`: servidor HTTP que responde consultas de ruta punto a punto (`/route/v1/driving/{coords}`) con tiempos de respuesta típicos < 5 ms para grafos regionales.

El desacople entre preprocesamiento (offline, una sola vez) y consulta (online, por época) es esencial para el simulador: la fase de contracción se ejecuta una vez sobre el extracto cartográfico, y cada época t del algoritmo rolling horizon emite $O(|\mathcal{C}| \cdot |S|)$ consultas al servidor, que se resuelven individualmente en tiempo constante amortizado.

La sustitución de la distancia euclidiana por tiempos OSRM tiene tres consecuencias directas sobre las heurísticas de la Sección 2.2:

- Los tiempos de viaje dejan de ser simétricos: $T(a, b) \neq T(b, a)$ por sentidos de circulación.
- La desigualdad triangular se preserva entre pares de nodos del grafo vial en ausencia de restricciones de giro, pero puede no cumplirse de forma estricta entre ubicaciones *snapped* cuando intervienen sentidos de circulación opuestos o restricciones de giro complejas; OSRM gestiona estos casos devolviendo siempre la ruta mínima factible entre los nodos de inicio y fin, de modo que la heurística opera sobre una matriz de costos consistente.
- El factor de conversión γ de Reyes et al. [13] se elimina: los tiempos provienen directamente de las velocidades y topología reales.

La Sección 3.1 detalla la configuración concreta del contenedor Docker, el perfil de velocidades utilizado y la integración con el módulo `getrouteOSMR.py`.

2. 4. Generación estocástica de demanda y métricas

La evaluación de políticas de despacho requiere instancias de prueba cuyas llegadas reproduzcan la variabilidad temporal observada en operaciones reales. Esta sección presenta el marco probabilístico que sustenta la generación de demanda y formaliza las familias de indicadores de desempeño utilizadas para comparar las políticas.

2. 4.1. Proceso de Poisson no homogéneo

Un proceso de Poisson no homogéneo (PPNH) es un proceso de conteo $\{N(t), t \geq 0\}$ en el que el número de eventos en un intervalo $(t_1, t_2]$ sigue una distribución de Poisson con parámetro igual a la integral de una función de intensidad $\lambda(\cdot)$ [29]:

$$N(t_2) - N(t_1) \sim \text{Poisson}\left(\int_{t_1}^{t_2} \lambda(u)\, du\right), \quad \lambda(u) > 0 \tag{2.10}$$

con incrementos independientes en intervalos disjuntos. A diferencia del proceso de Poisson homogéneo, la tasa $\lambda(t)$ varía con el tiempo, lo que permite capturar los picos y valles de demanda característicos de los servicios de entrega de comida.

Propiedad de discretización. Cuando la intensidad se integra sobre intervalos unitarios $[t, t+1)$, el conteo en cada intervalo hereda la distribución de Poisson:

$$N_t \sim \text{Poisson}\left(\int_t^{t+1} \lambda(u)\, du\right) \approx \text{Poisson}\big(\lambda(t)\big),$$

aproximación válida si λ varía lentamente dentro del intervalo. El error relativo del reemplazo $\int_t^{t+1} \lambda(u)\, du \approx \lambda(t)$ está acotado por $|\lambda'(t)|/(2\lambda(t))$ por expansión de Taylor de primer orden, magnitud despreciable para los parámetros adoptados ($\sigma = 45$ min, ver Sección 3.2). Este resultado justifica el muestreo minuto a minuto empleado en la Sección 3.2.

2. 4.2. Función de intensidad bimodal

La demanda de los servicios de entrega de comida suele concentrarse en dos picos diarios (almuerzo y cena). Como no se dispone de datos de entrega de comida para La Paz, este trabajo toma la forma del perfil diario del dataset LaDe [18], que proviene de la operación de paquetería de última milla de Cainiao: en Shanghai, la jornada de los repartidores va de 8:00 a 19:00 y la recolección de paquetes presenta picos a las 9:00 y a las 17:00. Una representación parsimoniosa de esta estructura bimodal es la superposición de una intensidad base constante y dos componentes gaussianas:

$$\lambda(t) = \lambda_0 + A_m \exp\left(-\frac{(t-\mu_m)^2}{2\sigma^2}\right) + A_e \exp\left(-\frac{(t-\mu_e)^2}{2\sigma^2}\right) \tag{2.11}$$

donde λ_0 es la intensidad base, μ_m y μ_e son los centros temporales de los picos matutino y vespertino, A_m y A_e las amplitudes respectivas, y σ la dispersión temporal compartida. Los centros y la ventana operativa se fijan a partir del perfil de LaDe [18]; la Sección 3.2 detalla los valores adoptados y el procedimiento de muestreo implementado para La Paz.

La elección del PPNH con perfil gaussiano bimodal se justifica por tres razones:

1. **Parsimonia:** seis parámetros $(\lambda_0, A_m, A_e, \sigma, \mu_m, \mu_e)$ capturan la esencia del patrón diario sin sobreajuste.

2. **Flexibilidad:** escalar λ_0 permite variar el volumen total de órdenes manteniendo la forma temporal.

3. **Tratabilidad:** la propiedad de discretización permite generar instancias mediante un simple muestreo de Poisson por minuto, sin necesidad de algoritmos de adelgazamiento (*thinning*).

Es preciso reconocer que la forma bimodal tomada de LaDe [18] corresponde a la recolección de paquetes en China y no a la demanda de comida; su transferibilidad al contexto de entrega de comida en La Paz constituye un supuesto operativo del modelo. Los centros μ_m y μ_e del perfil (Sección 3.2) conservan la ubicación de los picos de LaDe y no corresponden a los horarios habituales de comida en México (alrededor de las 14:00 y las 20:00); ni la bimodalidad, ni la posición de los picos, ni la dispersión σ se calibran con datos propios al no existir trazas operativas para La Paz. Esta limitación se discute en el Capítulo 5 como línea de trabajo futuro.

2. 4.3. Métricas de desempeño: marco de evaluación

La Sección 2.1 introdujo las métricas primarias del MDRP definidas por Reyes et al. [13]: Click-to-Door, Ready-to-Pickup, Ready-to-Door, utilización del courier y costo por orden. La importancia operativa de la métrica Click-to-Door ha sido reforzada por trabajos como el de Hildebrandt and Ulmer [30], quienes, citando estudios previos de servicio al cliente, señalan que la desviación respecto al tiempo de servicio esperado suele predecir la satisfacción mejor que el tiempo de servicio real, por lo que importan tanto la rapidez de la entrega como la precisión de su estimación (ETA). Para una evaluación cuantitativa rigurosa, estas métricas individuales (por orden o por courier) se agregan en estadísticos de resumen y se organizan en cinco familias que cubren las perspectivas de los actores del sistema.

Estadísticos de agregación. Para cada métrica a nivel de orden (e.g., CtD_o), se calculan:

- **Media** $\bar{m} = \frac{1}{n}\sum_{o=1}^{n} m_o$, que resume el comportamiento central.
- **Percentiles** P_{50}, P_{90}, P_{95}: el percentil 90 es particularmente relevante porque captura la experiencia de los clientes en la cola de la distribución, donde se concentran las entregas con mayor demora.

Familias de indicadores. Las métricas se clasifican en cinco dimensiones:

1. **Calidad de servicio.** CtD (media, P_{50}, P_{90}, P_{95}), RtP (media, P_{90}), RtD (media, P_{90}), tasa de cumplimiento SLA:

$$\text{SLA} = \frac{|\{o : \text{CtD}_o \leq \tau\}|}{|\mathcal{O}_{del}|} \times 100\,\% \tag{2.12}$$

   y exceso promedio respecto al objetivo $\bar{\Delta} = \frac{1}{n}\sum_o \text{máx}(0, \text{CtD}_o - \tau)$.

2. **Cobertura.** Fracción de órdenes no entregadas al cierre de la simulación.

3. **Eficiencia operativa.** Pedidos por repartidor por hora, agrupamientos por hora, utilización (% del turno conduciendo o en servicio), distancia total y distancia por pedido.

4. **Eficiencia de agrupamiento.** Tamaño promedio de agrupamiento y proporción de órdenes en agrupamientos múltiples, indicadores que reflejan el grado de consolidación logrado por la política de *bundling*.

5. **Costo.** Compensación total, costo por pedido y fracción de repartidores con compensación mínima garantizada, derivadas del modelo de compensación dual de la Ecuación 3.8.

El presente trabajo implementa 21 indicadores distribuidos en estas cinco familias; la Sección 3.7 detalla la definición operativa de cada uno y la Tabla 3.4 los enumera con su formulación abreviada.

Capítulo 3

Desarrollo

Este capítulo describe la implementación computacional del entorno de simulación para el MDRP en La Paz, B.C.S. Se documenta la arquitectura del sistema, la generación de instancias sintéticas, el motor de simulación por épocas, las dos políticas de despacho implementadas (FCFS y *Rolling Horizon*), el diseño experimental y las métricas de evaluación.

3. 1. Arquitectura del sistema

El sistema se organiza en tres capas funcionales desacopladas: (1) una capa de enrutamiento basada en OSRM contenerizado, (2) una capa de simulación implementada en Python, y (3) una capa de análisis que recolecta y procesa los indicadores de desempeño. La Figura 3.1 ilustra la interacción entre estos componentes.

3. 1.1. Capa de enrutamiento: OSRM contenerizado

La capa de enrutamiento proporciona tiempos y distancias de viaje realistas sobre la red vial de La Paz. Se implementa mediante OSRM (*Open Source Routing Machine*) v5.27.1 [31], cuya inicialización y configuración se definen en un archivo Docker Compose.

Preprocesamiento de la red vial

El proceso de construcción de la red vial sigue la cadena de preprocesamiento estándar de OSRM. A partir del extracto cartográfico de OpenStreetMap para Baja California Sur

(`bcs-latest.osm.pbf`), se ejecutan dos etapas secuenciales:

1. **Extracción** (`osrm-extract`): Procesa el archivo `.pbf` y genera una representación del grafo vial, aplicando el perfil de velocidades del vehículo (`car.lua`). Este perfil define las velocidades máximas permitidas por tipo de vía (autopista, avenida, calle residencial), así como las restricciones de giro y sentido.

2. **Contracción** (`osrm-contract`): Aplica el algoritmo de *Contraction Hierarchies* [27] sobre el grafo extraído, generando los archivos de índice `bcs-latest.osrm.*` que codifican la jerarquía de consulta rápida. Estos archivos son cargados directamente por `osrm-routed` al iniciar el contenedor Docker, exponiendo la API REST en `localhost:5000`.

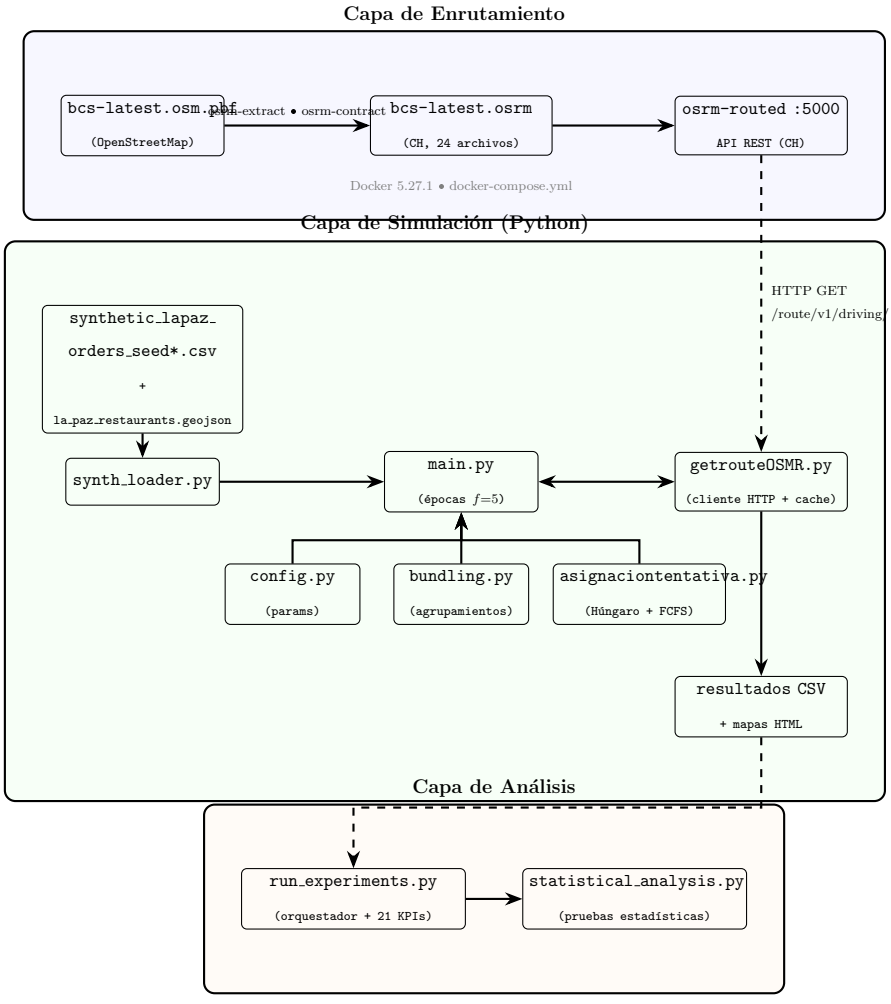


Figura 3.1: Arquitectura de tres capas del entorno de simulación MDRP-BCS.

Estos archivos preprocesados se incluyen en el repositorio bajo el directorio `osrm_data/`,

evitando que los usuarios deban repetir el preprocesamiento (que requiere aproximadamente 10–15 minutos y el archivo `.pbf` original).

Es importante señalar que un archivo `.pbf` constituye una version congelada de la cartografía en un momento dado. Aunque OpenStreetMap se actualiza de forma continua incorporando nuevas calles, colonias y modificaciones viales por medio de contribuciones colaborativas, los archivos preprocesados no reflejan estos cambios automáticamente. Si se desea incorporar actualizaciones cartográficas (por ejemplo, la apertura de un nuevo tramo vial en La Paz), es necesario descargar un `.pbf` más reciente y repetir la cadena completa de preprocesamiento (`osrm-extract` → `osrm-contract` → reinicio de `osrm-routed`). Para garantizar la reproducibilidad de los experimentos, el presente trabajo congela la versión del mapa utilizada; el extracto empleado corresponde a la descarga de Geofabrik para Baja California Sur con fecha de corte diciembre de 2025. Cualquier replicación futura debe utilizar este mismo archivo o documentar explícitamente la fecha del nuevo extracto.

Despliegue con Docker Compose

El servicio de enrutamiento se despliega de forma declarativa mediante un archivo `docker-compose.yml` que encapsula toda la configuración necesaria:

Listing 3.1: Definición del servicio OSRM en `docker-compose.yml`.

```
services:
  osrm:
    image: ghcr.io/project-osrm/osrm-backend:v5.27.1
    container_name: mdrp-osrm
    ports:
      - "5000:5000"
    volumes:
      - ./osrm_data:/data:ro
    command: osrm-routed --algorithm ch /data/bcs-latest.osrm
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f",
             "http://localhost:5000/route/v1/driving/-110.31,24.14;-110.30,24.15"]
      interval: 30s
      timeout: 10s
```

```
    retries: 3
```

Esta configuración garantiza tres propiedades esenciales para la reproducibilidad:

- **Aislamiento del entorno:** La imagen Docker `ghcr.io/project-osrm/osrm-backend:v5.27.1` fija la versión exacta del motor de enrutamiento, eliminando discrepancias por versiones de librerías del sistema operativo anfitrión.
- **Inmutabilidad de datos:** El volumen se monta en modo de solo lectura (`:ro`), garantizando que los archivos de la red vial preprocesada no sean modificados accidentalmente durante la ejecución.
- **Verificación de salud:** El `healthcheck` ejecuta una consulta de enrutamiento real cada 30 segundos, asegurando que el servicio esté operativo antes de que el simulador inicie las consultas. La coordenada de prueba $(-110.31, 24.14)$ corresponde a un punto dentro de la zona urbana de La Paz.

El despliegue se reduce a un único comando: `docker-compose up -d`. El contenedor expone el puerto 5000 en `localhost`, donde la API REST de OSRM queda disponible para recibir consultas del simulador Python.

API REST y protocolo de consulta

El simulador consume la API de OSRM mediante solicitudes HTTP GET al *endpoint* `/route/v1/driving/`. El módulo `getrouteOSMR.py` implementa el cliente HTTP con las siguientes características:

- **Formato de consulta:** Las coordenadas se especifican como pares longitud/latitud separados por punto y coma. Por ejemplo, una ruta desde un restaurante hacia dos puntos de entrega se codifica como: `/route/v1/driving/lon1,lat1;lon2,lat2;lon3,lat3`
- **Respuesta JSON:** OSRM retorna la distancia total (metros), duración total (segundos), la geometría de la ruta codificada en formato *polyline*, y los detalles por tramo (*legs*) incluyendo las maniobras de cada paso.

- **Respaldo euclidiano:** Si la consulta OSRM falla (por desconexión o error de red), el sistema calcula una estimación basada en la distancia de Haversine a una velocidad constante configurable (por defecto 320 m/min $\approx$ 19.2 km/h). Este mecanismo, controlado por la variable de entorno `USE_EUCLIDEAN_ON_FAILURE`, garantiza que la simulación no se interrumpa. Y si sucede que es utilizado se registra para ser considerado en el análisis de resultados.

3. 1.2. Capa de simulación: estructura de módulos Python

La lógica de simulación se implementa en Python 3.12 y se estructura en seis módulos bajo el directorio `src/`, cada uno con una responsabilidad bien definida. La Tabla 3.1 resume la función de cada módulo.

Tabla 3.1: Módulos del simulador y su responsabilidad.

Módulo	**Responsabilidad**
`config.py`	Parámetros globales de la simulación
`synth_loader.py`	Carga de instancias y dimensionamiento de flota
`main.py`	Motor de simulación por épocas
`getrouteOSMR.py`	Cliente HTTP para OSRM
`bundling.py`	Generación de agrupamientos por restaurante
`asignaciontentativa.py`	Asignación courier-agrupamiento (Algoritmo Húngaro)

La Tabla 3.2 presenta los parámetros de configuración centralizados en `config.py`, que controlan el comportamiento de la simulación.

Tabla 3.2: Parámetros de configuración del simulador (`config.py`).

Parámetro	**Símbolo**	**Valor**	**Descripción**
Frecuencia de optimización	f	5 min	Intervalo entre épocas
Horizonte de asignación	Δ_u	20 min	Ventana de anticipación
Objetivo Click-to-Door	τ	40 min	Meta de calidad de servicio
Máximo Click-to-Door	τ_{max}	90 min	Límite absoluto de servicio
Tiempo de servicio	s_r, s_o	4 min	Por recolección y por entrega
Tamaño máximo de agrupamiento	B_{max}	4	Límite de órdenes por ruta
Penalización de frescura	β	0.1	Peso del retraso en asignación
Penalización de pickup	θ	0.1	Peso del retraso en recolección
Ventana de ready time	Δ_1	20 min	Anticipación de órdenes listas
Ventana de disponibilidad	Δ_2	20 min	Anticipación de couriers libres
Compromiso forzado	x	10 min	Forzado si orden espera $> x$
Pago por orden	p_1	\$10	Compensación variable
Pago mínimo por hora	p_2	\$15/h	Compensación garantizada

3. 1.3. Capa de análisis: generación de resultados

La tercera capa del sistema procesa las salidas crudas de la simulación y genera las métricas comparativas. Se compone de dos scripts complementarios:

- `run_experiments.py`: Orquesta la ejecución del pipeline experimental completo: invoca al generador de instancias sintéticas (`make_synth_orders.py`) cuando se requieren múltiples semillas, ejecuta las simulaciones FCFS y *Rolling Horizon*, calcula 21 KPIs comparativos y consolida los resultados. Define cuatro niveles de relación órdenes/courier (18:1, 26:1, 35:1 y 52:1), ejecuta ambas políticas en cada escenario, y genera tablas consolidadas (`experiment_comparison.csv`, `experiment_summary.csv`) así como metadatos de la ejecución del experimento (`experiment_metadata.json`). Adicionalmente, soporta la ejecución con múltiples semillas aleatorias (`--seeds`) para análisis de robustez y la variación de parámetros algorítmicos (`--bundle-sizes`).

- `statistical_analysis.py`: Aplica las pruebas inferenciales definidas en la Sección 3. 6.4 (Wilcoxon signed-rank pareado a nivel semilla, prueba ómnibus de Friedman entre escenarios y rank-biserial pareado como tamaño de efecto) sobre los KPIs generados por `run_experiments.py`, produciendo los archivos `statistical_summary.csv` y `cross_seed_aggregation.csv`.

La separación en tres capas independientes permite modificar cualquier componente sin afectar a los demás. Por ejemplo, sustituir OSRM por otro motor de enrutamiento solo requiere modificar `getrouteOSMR.py`, sin alterar la lógica de simulación ni el cálculo de métricas. De manera análoga, agregar nuevos escenarios experimentales no requiere cambios en el simulador ni en la infraestructura de enrutamiento.

3. 2. Generación de instancias sintéticas

Dado que no se dispone de registros históricos de operaciones de entrega de comida en La Paz, B.C.S., el sistema genera instancias sintéticas calibradas que replican los patrones temporales y geoespaciales documentados en la literatura. El proceso se divide en dos etapas: (1) la generación de órdenes (`make_synth_orders.py`) y (2) la carga y dimensionamiento de flota (`synth_loader.py`).

3. 2.1. Generacion temporal de órdenes

La llegada de ordenes se modela mediante un **proceso de Poisson no homogeneo** [29] con tasa variable $\lambda(t)$, con la forma diaria del dataset LaDe [18]. Se adopta la función de intensidad bimodal definida en la Ec. (2.11) (§2.4.2), con los siguientes parámetros, que reproducen los picos de recolección de paquetes documentados en LaDe [18]: $\lambda_0 = 0.6$ órdenes/minuto, $\mu_m =$ 09:00, $\mu_e =$ 17:00, $A_m = 2.4$, $A_e = 2.0$, y $\sigma = 45$ min. Los centros μ_m y μ_e reproducen la ubicación de los picos de LaDe dentro de la ventana operativa y no los horarios de comida habituales en México (Sección 2.4.2).

El perfil de intensidad se discretiza en intervalos de un minuto sobre la ventana operativa de 11 horas (08:00–19:00, $T = 660$ min). Aplicando la aproximación por discretización descrita en §2.4.1, para cada minuto t el número de órdenes generadas se muestrea de una distribución de Poisson con parámetro $\lambda(t)$:

$$N(t) \sim \text{Poisson}(\lambda(t)), \quad t = 0, 1, \ldots, T-1 \tag{3.1}$$

El perfil se escala linealmente para que el total esperado de órdenes coincida con el objetivo configurado (por defecto 1,000). Dentro de cada minuto, el segundo exacto de llegada se asigna uniformemente en $[0, 60)$, proporcionando resolución temporal a nivel de segundo.

3. 2.2. Tiempo de preparación

Cada orden recibe un **tiempo de preparación** T_{prep} que determina cuándo la comida estará lista para ser recogida por el repartidor. Este tiempo se modela como una variable aleatoria normal truncada:

$$T_{prep} \sim \text{máx}\big(5, \mathcal{N}(\mu_{prep}, \sigma^2_{prep})\big), \quad \mu_{prep} = 12 \text{ min}, \quad \sigma_{prep} = 3 \text{ min} \tag{3.2}$$

Los valores $\mu_{prep} = 12$ y $\sigma_{prep} = 3$ son un supuesto del generador. Las instancias publicadas del MDRP, derivadas de datos reales [13, 14], incluyen tiempos de preparación, pero estos parámetros no se calibraron contra ellas ni contra datos de La Paz. El truncamiento inferior en 5 minutos evita tiempos de preparación irrealistamente cortos. El *ready time* de cada orden se calcula como:

$$t_{ready} = t_{created} + T_{prep} \tag{3.3}$$

donde $t_{created} = a_o$ es el instante de colocación de la orden (`placement_time`). Este ready time constituye una restricción dura en el modelo de simulación: ningún courier puede recoger la orden antes de t_{ready}, conforme a la definición del MDRP en Reyes et al. [13].

3. 2.3. Distribución geoespacial

Las ubicaciones de restaurantes y clientes se generan mediante muestreo uniforme dentro de un polígono que delimita la zona urbana de La Paz. Este polígono se define por cuatro vértices que encierran el área de mayor densidad poblacional y comercial:

$$\mathcal{P} = \text{Polígono}\left(\begin{bmatrix}24.124\\-110.312\end{bmatrix}, \begin{bmatrix}24.133\\-110.336\end{bmatrix}, \begin{bmatrix}24.159\\-110.310\end{bmatrix}, \begin{bmatrix}24.147\\-110.293\end{bmatrix}\right) \tag{3.4}$$

El muestreo utiliza un método de rechazo (*rejection sampling*): se generan puntos uniformemente en el rectángulo delimitador del polígono y se descartan aquellos que caen fuera de $\mathcal{P}$ (por ejemplo, sobre la bahía de La Paz). Esto garantiza que todas las ubicaciones generadas correspondan a puntos terrestres dentro de la mancha urbana. El muestreo es uniforme dentro de $\mathcal{P}$: no se pondera por densidad de población ni por la ubicación real de los comercios.

Asignación restaurante-cliente

Para cada punto de entrega (cliente), se selecciona un restaurante cuya distancia geodésica se encuentre en el rango $[2, 5]$ km. Este rango es un supuesto del generador para trayectos urbanos de entrega de comida; LaDe [18] no reporta distancias restaurante–cliente (solo indica que los paquetes consecutivos de una ruta suelen estar a menos de 1 km). La distancia se aproxima mediante:

$$d \approx \sqrt{(\Delta\phi)^2 + (\Delta\psi)^2} \times 111 \text{ km/grado} \tag{3.5}$$

donde $\Delta\phi$ y $\Delta\psi$ son las diferencias en latitud y longitud respectivamente. Si múltiples restaurantes caen dentro del rango, se elige uno al azar. Si ninguno cumple la restricción, se selecciona el restaurante más cercano que exceda la distancia mínima de 2 km como mecanismo de respaldo.

La aproximación anterior usa 111 km por grado también en longitud, sin el factor $\cos\phi$ (a 24° N un grado de longitud mide $\approx$ 101 km), y el polígono urbano es pequeño, por lo que el rango $[2, 5]$ km es nominal. Medidas con la fórmula de Haversine sobre las 10 184 órdenes de las diez semillas, las distancias en línea recta restaurante–cliente tienen mediana de 2.30 km (P5 = 1.90 km, P95 = 3.23 km, máximo 4.20 km); el 85.5 % cae dentro de $[2, 5]$ km.

Fuente de datos de restaurantes

Las ubicaciones base de los restaurantes se obtienen del archivo `la_paz_restaurants.geojson`, que contiene 25 establecimientos de comida en La Paz extraídos de OpenStreetMap [24] mediante consultas a la API de Overpass. Para garantizar la consistencia espacial con la zona de operación, las coordenadas reales se descartan y se reemplazan por ubicaciones generadas mediante muestreo uniforme dentro del polígono urbano $\mathcal{P}$, preservando únicamente el conteo de 25 establecimientos.

3. 2.4. Dimensionamiento automático de flota

El módulo `synth_loader.py` transforma el CSV de órdenes generado en objetos de simulación (`Order`, `Courier`, `Restaurant`) y calcula automáticamente el tamaño de la flota de couriers. La fórmula de dimensionamiento es:

$$n_{couriers} = \text{máx}\left(n_{min},\ \left\lceil \frac{|\mathcal{O}|}{r} \right\rceil\right) \tag{3.6}$$

donde $|\mathcal{O}|$ es el número total de órdenes, r es el ratio objetivo de órdenes por courier (`orders_per_courier`, por defecto $r = 18$), y n_{min} es un piso mínimo de flota (`min_couriers`, $n_{min} = 10$ en los experimentos del Capítulo 4; el valor por defecto del módulo `synth_loader.py` es 15 y se sobrescribe a 10 desde `run_experiments.py`). Este mecanismo permite variar el nivel de saturación de la flota: valores altos de r producen flotas más pequeñas y mayor carga por courier, mientras que valores bajos generan flotas holgadas con baja utilización.

Configuración de turnos

Todos los couriers comparten un turno uniforme que se calcula a partir de los datos:

- **Inicio del turno:** 15 minutos antes de la primera orden colocada, permitiendo que los couriers estén disponibles cuando arriben las primeras solicitudes.
- **Fin del turno:** 1 hora después del último *ready time*, garantizando tiempo suficiente para completar las entregas finales.
- **Ubicación inicial:** Todos los couriers parten del centroide geográfico de los restaurantes, representando un punto de espera central antes de la primera asignación.

Reproducibilidad

El generador acepta una semilla aleatoria (`--seed`, por defecto 2025) que inicializa el generador de números pseudoaleatorios de NumPy (`numpy.random.default_rng`). Esto garantiza que, dada la misma semilla y el mismo objetivo de órdenes, se produzca una instancia idéntica en cualquier máquina. El script `run_experiments.py` aprovecha esta propiedad para ejecutar múltiples réplicas con semillas distintas, habilitando el análisis de variabilidad estadística entre instancias.

3. 3. Motor de simulación

El motor de simulación (`main.py`) implementa un ciclo discreto dirigido por épocas que avanza el reloj virtual en incrementos de $f = 5$ min. En cada época, el motor ejecuta cuatro fases secuenciales: activación, revelación, asignación y actualización.

3. 3.1. Ciclo de simulación por épocas

Sea t el tiempo de simulación actual. El ciclo se inicia en t_0 (15 min antes de la primera orden) y termina en t_{end} (1 h después del último *ready time*). En cada iteración se ejecuta:

1. **Activación de couriers:** Se recorre la lista completa de couriers y se incorporan al conjunto de couriers activos $\mathcal{C}_{act}$ aquellos cuyo *on-time* $e_c \leq t$:

$$\mathcal{C}_{act}(t) = \{c \in \mathcal{C} \mid e_c \leq t\} \tag{3.7}$$

2. **Revelación de órdenes:** Las órdenes cuyo *placement time* $a_o \leq t$ se extraen de la cola ordenada (`deque`) y pasan al estado `ready`, quedando disponibles para asignación. Cada orden se registra en la lista de órdenes pendientes de su restaurante.

3. **Asignación:** En cada época (el reloj avanza en pasos de f, sin importar el minuto en que inicie la simulación) se activa la lógica de despacho (FCFS o *Rolling Horizon*, según la variable de entorno `FCFS_POLICY`). La selección de política se realiza en tiempo de ejecución, permitiendo ejecutar ambas sobre la misma instancia sin modificar código.

4. **Actualización de rutas:** Se verifica si algún courier ha completado su ruta activa ($t \geq t_{completion}$). Para rutas finalizadas, se registran los tiempos de pickup y delivery de cada orden —calculados con la temporización de la Sección 3. 3.4—, se actualiza la ubicación del courier al último waypoint, se acumulan distancia y tiempo de conducción, y se libera al courier para futuras asignaciones.

Finalmente, el reloj avanza $t \leftarrow t + f$ y el ciclo se repite. Al terminar la simulación, se calcula la compensación final de cada courier.

Como la actualización ocurre después de la asignación, un courier que termina su ruta en el intervalo $(t - f, t]$ se libera en la actualización de la época t y recibe su siguiente ruta en la época $t + f$: entre el fin de una ruta y el inicio de la siguiente transcurren entre 5 y 10 min. Esta espera afecta más a la política que realiza más viajes; su efecto sobre los resultados se evalúa en la Sección 4. 6.

3. 3.2. Modelos de datos: órdenes, couriers y restaurantes

El motor define tres clases que encapsulan el estado de las entidades durante la simulación:

- **Order:** Almacena `placement_time`, `ready_time`, `dropoff_loc`, el restaurante de origen, y campos mutables: `status` (`pending` → `ready` → `assigned` → `delivered`), `pickup_time`, `delivery_time`, `courier_id` y `bundle_size`. Expone métodos para calcular el Click-to-Door (CtD $= t_{delivery} - t_{placement}$) y el Ready-to-Pickup (RtP $= t_{pickup} - t_{ready}$).

- **Courier:** Almacena `on_time`, `off_time`, ubicación actual, ruta activa (`current_route`) e historial de rutas completadas. Acumula métricas operativas: órdenes entregadas, distancia total (km), tiempo de conducción (min), tiempo de servicio (min) y agrupamientos

recogidos. El método `final_compensation()` implementa el modelo de pago dual:

$$\text{Compensación} = \max(p_1 \cdot n_{orders},\ \ p_2 \cdot h_{shift}) \tag{3.8}$$

donde $p_1 = \$10$/orden, $p_2 = \$15$/h, y h_{shift} es la duración del turno en horas.

- **Restaurant:** Almacena identificador, ubicación (lat, lon) y la lista de órdenes pendientes. Sirve como punto de agrupación para el módulo de *bundling*.

3. 3.3. Máquina de estados de una orden

Cada orden transita por cuatro estados durante su ciclo de vida:

1. `pending`: La orden ha sido colocada pero su *placement time* aún no ha sido alcanzado por el reloj de simulación. La orden permanece en la cola ordenada.

2. `ready`: El reloj ha alcanzado o superado a_o. La orden se ha revelado al sistema y está disponible para asignación. Se registra en la lista de órdenes pendientes de su restaurante.

3. `assigned`: La orden ha sido asignada a un courier y forma parte de su ruta activa. En la política FCFS, esta transición ocurre de forma explícita al seleccionar al courier más cercano. En la política *Rolling Horizon*, la transición ocurre cuando el agrupamiento recibe un compromiso final; a partir de ese momento la orden ya no puede incluirse en otro agrupamiento.

4. `delivered`: Un courier ha completado la entrega. Se registran t_{pickup} y $t_{delivery}$, y la orden se mueve a la lista de órdenes entregadas para el cálculo posterior de KPIs.

Las transiciones `pending` → `ready` ocurren en la fase de revelación de cada época. La transición `ready` → `assigned` ocurre en la fase de asignación, y la transición `assigned` → `delivered` ocurre en la fase de actualización, cuando el courier completa su ruta. Bajo la política *Rolling Horizon* con compromiso parcial (inbound, véase Sección 3.5.4), la orden permanece en `ready` mientras el courier se desplaza al restaurante: solo transita a `assigned` cuando el compromiso final se concreta en una época posterior, preservando así la flexibilidad de reoptimizar el agrupamiento.

3. 3.4. Temporización de rutas

Ambas políticas calculan los instantes de recolección y entrega con las reglas de Reyes et al. [13] (Sección 2.1.3), implementadas en el módulo `schedule.py`. Una ruta final asignada en el instante t recorre la ubicación del courier, el restaurante r y los clientes $\ell_{o_1}, \ldots, \ell_{o_n}$ del agrupamiento, en ese orden. Sean T_0 la duración OSRM del tramo courier $\to r$ y T_k la del tramo que llega al cliente k. Entonces:

$$t_{arr,r} = t + T_0, \tag{3.9}$$

$$t_{pickup} = \max\Big(\max_{o \in B} e_o,\ t_{arr,r} + \tfrac{s_r}{2}\Big), \tag{3.10}$$

$$t_{del,o_1} = t_{pickup} + \tfrac{s_r}{2} + T_1 + \tfrac{s_o}{2}, \tag{3.11}$$

$$t_{del,o_k} = t_{del,o_{k-1}} + \tfrac{s_o}{2} + T_k + \tfrac{s_o}{2}, \quad k = 2, \ldots, n, \tag{3.12}$$

y el courier queda libre en $t_{del,o_n} + s_o/2$. Si la comida no está lista cuando el repartidor llega, este espera en el restaurante. En consecuencia, ninguna orden se recoge antes de su *ready time* ni se entrega antes de recogerse, y cada orden de un agrupamiento tiene su propio instante de entrega. Las mismas expresiones alimentan el peso de asignación (Ec. 3.18) y la clasificación por prioridad, de modo que la política evalúa las rutas con los mismos tiempos con que después se ejecutan.

3. 3.5. Integración con OSRM

El motor se comunica con el servicio de enrutamiento a través de `getrouteOSMR.py`, que actúa como capa de abstracción entre la simulación y la API REST. Cada vez que un módulo de asignación (FCFS o *Rolling Horizon*) necesita evaluar una ruta candidata,
invoca `get_route_details(origin, waypoints)`, que:

1. Verifica si la consulta existe en la caché en memoria (`_osrm_cache`), indexada por la tupla `(origin, waypoints)`.
2. Si hay *cache hit*, retorna el resultado almacenado sin consulta HTTP.
3. Si hay *cache miss*, construye la URL con coordenadas en formato `lon,lat` y realiza la solicitud GET al contenedor OSRM.

4. Si la solicitud falla, aplica el respaldo euclidiano (Haversine a 320 m/min).

5. Almacena el resultado en caché para consultas futuras.

La respuesta de OSRM se estructura como un diccionario con campos `distance` (metros), `duration` (segundos), `geometry` (polyline codificado) y `legs` (tramos individuales). Estos valores alimentan directamente el cálculo de tiempos de pickup, delivery y los KPIs de distancia recorrida.

3. 4. Política FCFS (*baseline*)

La política *First-Come, First-Served* (FCFS) sirve como línea base contra la cual se evalúa el desempeño de la política *Rolling Horizon*. Implementa una lógica de despacho reactiva sin agrupamiento ni optimización global: cada orden se asigna individualmente al courier disponible más cercano en el momento de la decisión.

3. 4.1. Lógica de asignación

En cada época de asignación, la política FCFS toma todas las órdenes reveladas que aún no tienen courier (estado `ready`), sin importar si la comida ya está lista, las ordena por instante de colocación a_o y ejecuta para cada una el siguiente procedimiento. En la práctica, cada orden se despacha en la primera época posterior a su colocación en la que haya un courier libre:

1. **Filtrar couriers disponibles:** Se construye el conjunto de couriers sin ruta activa cuyo turno no haya expirado:

$$\mathcal{C}_{disp}(t) = \{c \in \mathcal{C}_{act}(t) \mid c.current_route = \varnothing \ \wedge \ l_c > t\} \tag{3.13}$$

   donde l_c es el fin del turno del courier (`off_time`).

2. **Selección por proximidad:** Para cada courier disponible se calcula la distancia euclidiana a la ubicación del restaurante de la orden:

$$d(c, o) = \sqrt{(\phi_c - \phi_r)^2 + (\psi_c - \psi_r)^2} \tag{3.14}$$

   donde (ϕ_c, ψ_c) es la ubicación actual del courier y (ϕ_r, ψ_r) la del restaurante. Se selecciona $c^* = \arg\min_{c \in \mathcal{C}_{disp}} d(c, o)$. Esta expresión opera en grados sobre la latitud y longitud y

se usa únicamente como ordenador para elegir al courier; el tiempo de viaje efectivo se obtiene a continuación con OSRM. Como el ordenamiento no considera la red vial, el courier elegido puede no ser el que llega antes al restaurante, lo que puede desfavorecer a FCFS cuando los tiempos provienen de OSRM (Sección 4. 7).

3. **Consulta de ruta:** Se solicita a OSRM la ruta completa $c^*.location \to r_o \to \ell_o$, donde r_o es la ubicación del restaurante y ℓ_o el punto de entrega del cliente. Los instantes de recolección y entrega se calculan con la temporización de la Sección 3. 3.4: si el courier llega antes de que la comida esté lista, espera en el restaurante.

4. **Compromiso inmediato:** Se asigna la ruta al courier como compromiso `final` (sin etapa parcial) y la orden pasa al estado `assigned`. El courier queda ocupado hasta salir del domicilio del cliente.

Al procesarse en orden de colocación, las órdenes más antiguas obtienen primero los couriers más cercanos, que es el criterio *first-come, first-served*. Las órdenes que no encuentran courier libre permanecen en espera y se reintentan en la época siguiente.

3. 4.2. Características y limitaciones

La política FCFS presenta las siguientes propiedades que la distinguen de la política *Rolling Horizon*:

- **Sin agrupamiento:** Cada orden genera una ruta individual (`bundle_size` $= 1$). El courier viaja al restaurante, recoge una sola orden y la entrega antes de quedar disponible para la siguiente asignación.

- **Sin optimización de matching:** La asignación se basa únicamente en proximidad geográfica instantánea, sin considerar el horizonte de asignación Δ_u, la frescura de las órdenes, ni la carga futura del sistema.

- **Compromiso directo:** No existe la etapa de compromiso parcial (*tentative assignment*). El courier se compromete de inmediato con la ruta completa pickup $\to$ delivery.

- **Activación:** La política se activa estableciendo la variable de entorno `FCFS_POLICY=1` antes de ejecutar la simulación. Sin esta variable, el motor utiliza *Rolling Horizon* por defecto.

Estas simplificaciones hacen de FCFS una línea base reactiva. Como RH difiere de FCFS en varios mecanismos a la vez —agrupamiento, asignación global, horizonte de anticipación, compromiso en dos etapas y grupos de prioridad—, la comparación evalúa las políticas completas y no atribuye la mejora a un mecanismo en particular; aislar la contribución de cada uno requiere un estudio de ablación como el de Reyes et al. [13], que se propone como trabajo futuro.

3. 5. Política *Rolling Horizon*

La política de *Rolling Horizon* (RH) constituye la heurística central de este trabajo, basada en el marco propuesto por Reyes et al. [13]. A diferencia de FCFS, RH agrupa órdenes del mismo restaurante en agrupamientos (*bundles*), resuelve un problema de asignación bipartita courier-agrupamiento mediante el Algoritmo Húngaro, y emplea un esquema de compromiso en dos etapas que permite reoptimizar asignaciones entre épocas. Esta sección describe los cuatro componentes secuenciales que se ejecutan en cada época de asignación.

3. 5.1. Cálculo del tamaño objetivo de agrupamiento

Al inicio de cada época, el sistema estima el tamaño objetivo de agrupamiento Z_t como el ratio entre las órdenes próximas a estar listas y los couriers que estarán disponibles en el horizonte inmediato [13]:

$$Z_t = \left\lceil \frac{|\{o \in \mathcal{O}(t) : t_{ready}(o) \leq t + \Delta_1\}|}{|\{c \in \mathcal{C}_{turno}(t) : e_c^{avail} \leq t + \Delta_2\}|} \right\rceil \tag{3.15}$$

donde $\mathcal{O}(t)$ es el conjunto de órdenes reveladas al tiempo t (estado `ready`), $\mathcal{C}_{turno}(t)$ es el conjunto de couriers activos cuyo turno se extiende al menos Δ_u más allá de t —libres u ocupados—, e_c^{avail} es el instante en que el courier c queda libre (fin de ruta actual, o t si está inactivo), $\Delta_1 = 20$ min es el horizonte de anticipación para órdenes listas y $\Delta_2 = 20$ min es el horizonte para disponibilidad de couriers. El valor se acota superiormente por `MAX_TARGET_BUNDLE_SIZE` $= 5$ para evitar agrupamientos irrealistamente grandes bajo picos de demanda. Si no hay couriers

disponibles en el horizonte, se utiliza cualquier courier activo en turno como denominador de respaldo; si aun así no hay couriers, $Z_t = 1$.

Este indicador adapta dinámicamente la consolidación de órdenes al nivel de saturación instantáneo: cuando la demanda supera la oferta de couriers, Z_t crece y se forman agrupamientos más grandes; cuando hay holgura, $Z_t \to 1$ y el comportamiento converge hacia asignaciones individuales similares a FCFS.

3. 5.2. Generación de agrupamientos por restaurante

Para cada restaurante con órdenes pendientes, el módulo `bundling.py` construye agrupamientos mediante un procedimiento de inserción paralela con mejora local (*parallel insertion with remove-reinsert improvement*):

1. **Filtrado:** Se seleccionan las órdenes del restaurante con estado `ready` cuyo *ready time* cae dentro del horizonte de asignación ($t_{ready} \leq t + \Delta_u$).

2. **Inicialización:** Se ordenan las órdenes por t_{ready} ascendente y se crean m_r agrupamientos vacíos, donde:
$$m_r = \min\left(\left\lceil \frac{n_{orders}}{Z_t} \right\rceil,\ |\mathcal{C}_{disp}|\right) \tag{3.16}$$
garantizando que no se generen más agrupamientos que couriers disponibles.

3. **Inserción:** Para cada orden, se evalúan todas las posiciones de inserción en todos los agrupamientos existentes. El costo de cada inserción candidata se calcula mediante:
$$\text{costo}(B, o, p) = T_{ruta}(B \oplus_p o) + \beta \cdot T_{servicio}(B \oplus_p o) \tag{3.17}$$
donde $B \oplus_p o$ denota la inserción de la orden o en la posición p del agrupamiento B, T_{ruta} es la duración OSRM de la ruta resultante, $T_{servicio}$ es el tiempo de servicio acumulado, y $\beta = 0.1$ es la penalización de frescura. Se selecciona la combinación (B^*, p^*) que minimiza el *incremento* de costo $\text{costo}(B, o, p) - \text{costo}(B)$, donde $\text{costo}(B)$ es el costo del agrupamiento actual (nulo si está vacío) [13]. Si un agrupamiento ya alcanza Z_t órdenes, solo se permite la inserción si mejora la eficiencia (tiempo promedio por orden). El tamaño máximo absoluto es `MAX_BUNDLE_SIZE` $= 4$.

4. **Mejora local (*remove-reinsert*):** Se ejecutan 2 iteraciones de un procedimiento de mejora: para cada orden en cada agrupamiento, se extrae temporalmente y se evalúa su reinserción en todas las posiciones de todos los agrupamientos, incluida la posición original. La orden se reinserta donde el incremento de costo es menor, de modo que cada orden permanece en exactamente un agrupamiento. Este paso permite corregir asignaciones subóptimas del procedimiento greedy inicial.

Los agrupamientos vacíos resultantes se descartan antes de pasar a la fase de asignación.

3. 5.3. Clasificación por prioridad y asignación bipartita

Una vez generados todos los agrupamientos de todos los restaurantes, se clasifican en tres grupos de prioridad según Reyes et al. [13]:

- Grupo i (urgente): El tiempo de entrega más temprano posible excede $a_o + \tau$ para la orden más antigua del agrupamiento, donde a_o es el *placement time* y $\tau = 40$ min es el objetivo de *Click-to-Door*. Ningún courier puede cumplir la meta de servicio.

- Grupo ii (retrasado): El agrupamiento no pertenece al Grupo i, pero ningún courier puede llegar al restaurante antes del *ready time* del agrupamiento. Existe retraso en la recolección pero la meta de entrega aún es alcanzable.

- Grupo iii (normal): El agrupamiento puede ser recogido y entregado dentro de los objetivos de servicio.

La asignación se resuelve en tres rondas secuenciales (i $\rightarrow$ ii $\rightarrow$ iii), priorizando los agrupamientos más urgentes. En cada ronda, se construye una matriz de costos $\mathbf{W} \in \mathbb{R}^{|\mathcal{C}_{disp}| \times |B_g|}$ donde cada entrada es el peso de matching:

$$w_{cj} = \frac{|B_j|}{T_{delivery}(c, B_j)} - \theta \cdot \left(t_{pickup}(c, B_j) - t_{ready}(B_j)\right) - P_g \tag{3.18}$$

donde $|B_j|$ es el número de órdenes en el agrupamiento j, $T_{delivery}(c, B_j)$ es el tiempo total de entrega en horas, $t_{pickup}(c, B_j)$ es el tiempo estimado de recolección, $t_{ready}(B_j)$ es el máximo *ready time* del agrupamiento, $\theta = 0.1$ penaliza el retraso en la recolección, y P_g es una penalización de

prioridad que refuerza la separación entre grupos: $P_g = 100$ si el par (c, B_j) excede τ_{max}, $P_g = 50$ si presenta retraso en la recolección, y $P_g = 0$ en caso contrario. Si un par courier-agrupamiento es infactible (sin ruta OSRM), se asigna un costo prohibitivo (10^9).

La matriz se convierte a costos ($c_{ij} = -w_{cj}$) y se resuelve mediante `scipy.optimize.linear_sum_assign` (SciPy 1.13), que resuelve el problema de asignación de forma exacta con una variante del algoritmo de Jonker–Volgenant, de complejidad $O(n^3)$. Como el Algoritmo Húngaro [23], obtiene una asignación de costo mínimo; por brevedad, en este documento se le llama Algoritmo Húngaro. Los couriers asignados a celdas con costo prohibitivo se descartan.

3. 5.4. Compromiso en dos etapas

Para cada par (c, B) seleccionado por el Algoritmo Húngaro, se aplica el compromiso aditivo en dos etapas (*two-stage additive commitment*) descrito en Reyes et al. [13]:

1. **Excepción de espera prolongada:** Si alguna orden del agrupamiento lleva lista más de $x = 10$ min ($t - t_{ready}(o) > x$), se fuerza un compromiso `final` independientemente de las demás condiciones. Esto evita que órdenes queden indefinidamente sin asignar.

2. **Compromiso final:** Si el courier puede llegar al restaurante dentro de la época actual ($t_{arr,r} \leq t+f$, con la duración OSRM del tramo courier → restaurante) **y** todas las órdenes del agrupamiento estarán listas dentro de la época ($t_{ready}(o) \leq t + f,\ \forall o \in B$), se asigna la ruta completa: courier → restaurante → entrega$_1$ → ⋯ → entrega$_n$, con los tiempos de la Sección 3. 3.4. El courier queda comprometido hasta salir del domicilio del último cliente.

3. **Compromiso parcial:** Si no se cumplen las condiciones anteriores, se asigna únicamente la ruta *inbound* (courier → restaurante). El courier comienza a desplazarse hacia el restaurante, pero en la siguiente época el sistema puede reoptimizar la asignación: actualizar el agrupamiento, cambiar de courier, o añadir/quitar órdenes.

4. **Ignorar:** Si no es posible calcular una ruta inbound (fallo de OSRM o conexión), la asignación se descarta y el agrupamiento queda disponible para la siguiente época.

Este mecanismo *lazy* de compromiso evita fijar prematuramente rutas con información parcial. Un courier con compromiso parcial aparece como ocupado (`current_route` $\neq \varnothing$), pero al

completar su tramo inbound en la fase de actualización, queda libre de nuevo, ubicado en el restaurante. En la práctica, esto proporciona una ventana de tiempo adicional para que más órdenes se revelen y puedan consolidarse en un agrupamiento más eficiente.

3. 5.5. Resumen del flujo RH por época

La Figura 3.2 resume el flujo completo de la política *Rolling Horizon* en cada época de asignación.

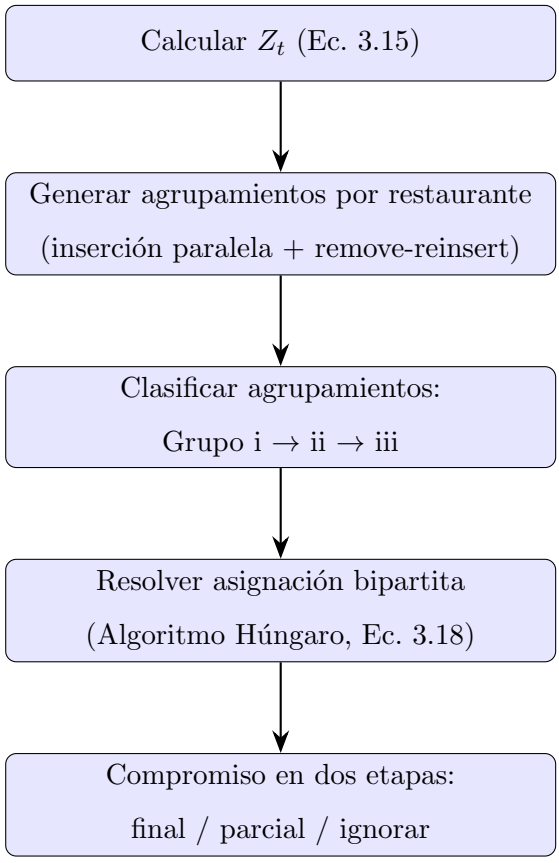


Figura 3.2: Flujo de la política *Rolling Horizon* en cada época de asignación.

3. 6. Diseño experimental

La evaluación comparativa de las políticas FCFS y *Rolling Horizon* se estructura como un experimento pareado sobre instancias sintéticas generadas según el proceso descrito en la Sección 3.2. El script `run_experiments.py` orquesta la ejecución completa.

3. 6.1. Variable de tratamiento: saturación de flota

La variable independiente es la *razón de saturación* r, definida como el número objetivo de pedidos por repartidor. A partir de r se dimensiona la flota según la Ecuación 3.6, manteniendo fijo el volumen de pedidos $|\mathcal{O}|$. Un valor alto de r produce una flota reducida con alta carga individual, mientras que un valor bajo genera capacidad holgada.

3. 6.2. Escenarios

Se definen cuatro escenarios que cubren desde operación holgada hasta saturación alta (Tabla 3.3). En todos ellos $n_{\text{mín}} = 10$; dado que incluso el escenario más saturado produce $\lceil 1\,038/52 \rceil = 20 > n_{\text{mín}}$, esta cota no resulta vinculante.

Tabla 3.3: Escenarios experimentales de saturación ($|\mathcal{O}| \approx 1\,038$ pedidos).

Escenario	Nivel	r	$\lvert\mathcal{C}\rvert$	Descripción
S1	Baja	18	58	Línea base; r igual al valor por defecto
S2	Moderada	26	40	Reducción moderada de flota
S3	Media	35	30	Punto medio de saturación
S4	Alta	52	20	Carga elevada por repartidor

Los valores de $|\mathcal{C}|$ de la Tabla 3.3 son nominales: como el número de órdenes varía entre semillas (de 988 a 1 047), la flota real de cada réplica también varía: 55–59 couriers en S1, 38–41 en S2, 29–30 en S3 y 19–21 en S4. El escenario S1 coincide con el valor por defecto de `TARGET_ORDERS_PER_COURIER` y sirve como referencia. El escenario S4 estresa deliberadamente el sistema para revelar los límites de cada política.

3. 6.3. Comparación pareada

Para cada combinación de escenario y semilla, ambas políticas se ejecutan sobre la *misma* instancia de pedidos y la *misma* flota de repartidores. La simulación se invoca en proceso (*in-process*): la función `_run_policy` ajusta el parámetro r del escenario y activa la política correspondiente mediante la variable de entorno `FCFS_POLICY` (presente → FCFS; ausente →

Rolling Horizon). Este diseño pareado elimina la variabilidad inter-instancia y aísla el efecto de la política de asignación.

3. 6.4. Plan de análisis estadístico

El diseño pareado de la subsección anterior condiciona la elección de las pruebas inferenciales que se aplicarán en el Capítulo 4. Cada semilla genera un par observacional (FCFS, RH) sobre la misma instancia, por lo que la unidad natural de análisis es la diferencia intra-semilla. Para evitar la pseudo-replicación [32] que resultaría de tratar cada orden como observación independiente —cuando todas las órdenes de una corrida comparten la misma realización aleatoria— cada métrica se agrega al nivel de semilla antes de la inferencia.

Como prueba primaria se adopta el test de Wilcoxon signed-rank pareado [33], no paramétrico, libre de supuestos de normalidad y apropiado para tamaños muestrales pequeños ($n \leq 10$ semillas). Se reporta la diferencia de medias entre semillas con intervalo de confianza t-Student al 95 % y el rank-biserial pareado como tamaño de efecto. Para evaluar si la magnitud del efecto varía entre regímenes de saturación se aplica la prueba ómnibus de Friedman [34] sobre la matriz semillas $\times$ escenarios de ΔCtD. Los resultados detallados se reportan en el Capítulo 4, Sección 4.5.

3. 6.5. Escenario de control euclidiano

Para separar el efecto de la política del efecto de la topología vial, el experimento completo se repite sustituyendo OSRM por la distancia en línea recta (Haversine) entre cada par de puntos, recorrida a velocidad constante (`run_experiments.py --euclidean`). La velocidad se fija en 651 m/min ($\approx$ 39 km/h), la velocidad media que OSRM asigna a los 10 184 trayectos restaurante–cliente de las diez instancias (`circuity.py`), de modo que ambos escenarios difieren solo en la geometría de las rutas y no en la rapidez de los repartidores. El rodeo que impone la red vial se cuantifica con el índice de circuidad $\kappa = d_{OSRM}/d_{Haversine}$ sobre los pares restaurante–cliente de las diez instancias (`circuity.py`). Un segundo control usa la velocidad reducida en ese factor ($651/\kappa \approx 472$ m/min, `--speed 472.1`), de modo que los trayectos en línea recta duran en promedio lo mismo que por la red; la diferencia entre este control y la red real aísla el efecto de la estructura de la red más allá de la longitud media de los viajes.

3. 6.6. Replicación y análisis de sensibilidad

El marco experimental admite dos ejes adicionales de variación:

- **Semillas múltiples.** Cada semilla genera una instancia distinta con la misma distribución subyacente, lo que permite estimar la variabilidad estadística de los indicadores a través de réplicas independientes.
- **Tamaño máximo de agrupamiento.** Se puede variar el parámetro `MAX_BUNDLE_SIZE` entre ejecuciones para evaluar su efecto sobre la eficiencia de la política RH, manteniendo el resto de la configuración constante.

El número total de ejecuciones de simulación es $2 \times |S| \times |Seeds| \times |B|$, donde $|S|$ es el número de escenarios, $|Seeds|$ las semillas y $|B|$ los valores de tamaño de agrupamiento evaluados; el factor 2 corresponde a las dos políticas.

3. 6.7. Almacenamiento de resultados

Cada corrida del marco genera un directorio con marca temporal bajo `results/experiments/`. Para cada escenario se almacenan cuatro archivos CSV (resultados de pedidos y repartidores, por política). Al finalizar todos los escenarios, el script produce:

- `experiment_comparison.csv`: tabla completa con los KPIs de ambas políticas por escenario y la mejora porcentual $\Delta\,\% = 100 \cdot (\text{RH} - \text{FCFS})/|\text{FCFS}|$.
- `experiment_summary.csv`: subconjunto de indicadores clave (CTD promedio, costo por pedido, pedidos por repartidor por hora, utilización, cumplimiento SLA y distancia por pedido).
- `experiment_metadata.json`: parámetros de la ejecución (semillas, escenarios, tamaños de agrupamiento, marca temporal).

3. 7. Métricas de evaluación

El desempeño de cada política se cuantifica mediante 21 indicadores clave (*KPIs*) agrupados en cinco dimensiones: calidad de servicio, cobertura, eficiencia operativa, eficiencia de agrupa-

miento y costo. Las definiciones se basan en las métricas propuestas por Reyes et al. [13] y se implementan en la función `calculate_kpis` de `run_experiments.py`.

3. 7.1. Calidad de servicio

El indicador principal es el *Click-to-Door* (CtD), definido como el tiempo transcurrido desde la colocación de la orden hasta su entrega al cliente:

$$\mathrm{CtD}_o = t_{delivery,o} - a_o$$

Se reportan la media y los percentiles 50, 90 y 95. Adicionalmente se calculan:

- *Ready-to-Pickup* (RtP): tiempo desde que la orden está lista en el restaurante hasta que el repartidor la recoge ($\mathrm{RtP}_o = t_{pickup,o} - t_{ready,o}$). Se reportan la media y el percentil 90.
- *Ready-to-Door* (RtD): tiempo desde la disponibilidad del pedido hasta su entrega ($\mathrm{RtD}_o = t_{delivery,o} - t_{ready,o}$). Se reportan la media y el percentil 90.
- **Cumplimiento SLA**: porcentaje de órdenes entregadas con $\mathrm{CtD}_o \leq \tau = 40$ min. Como complemento se reporta el cumplimiento sobre *todas* las órdenes generadas, que cuenta las no entregadas como incumplimientos y evita favorecer a la política que deja más pedidos sin entregar. Las diferencias entre políticas en métricas expresadas en porcentaje se reportan en puntos porcentuales (pp).
- **CTD Overage**: exceso promedio respecto al objetivo, $\frac{1}{n}\sum \max(0, \mathrm{CtD}_o - \tau)$.

3. 7.2. Cobertura

- **% Pedidos no entregados**: fracción de órdenes que no alcanzan el estado `delivered` al cierre de la simulación, calculada sobre el total de órdenes de la instancia.

3. 7.3. Eficiencia operativa

Estas métricas se derivan del CSV de repartidores, que registra distancia recorrida (km), tiempo de conducción (min), tiempo de servicio (min) y duración del turno (h) por repartidor. Como ambas políticas operan con la misma flota y los mismos turnos, los pedidos por repartidor

por hora solo difieren entre políticas en la medida en que difiere el número de pedidos entregados (cobertura).

- **Pedidos por repartidor por hora**: $\sum n_{del,c} \,/\, \sum h_{shift,c}$.
- **Agrupamientos por hora**: $\sum b_c \,/\, \sum h_{shift,c}$, donde b_c son los agrupamientos recogidos por el repartidor c.
- **Utilización** (%): fracción del turno que el repartidor estuvo conduciendo o en servicio (recolección y entrega), sin contar las esperas [13]: $100 \cdot \sum(t_{drive,c} + t_{serv,c}) \,/\, \sum h_{shift,c}$.
- **Distancia total** (km): suma de las distancias de ruta de todos los repartidores.
- **Distancia por pedido** (km): distancia total dividida entre el número de pedidos entregados.

3. 7.4. Eficiencia de agrupamiento

- **Tamaño promedio de agrupamiento**: media del campo `bundle_size` de las órdenes entregadas.
- **% Pedidos en multi-agrupamiento**: proporción de órdenes entregadas cuyo `bundle_size` > 1; refleja en qué medida la política logra consolidar entregas.

3. 7.5. Costo

El modelo de compensación sigue el esquema dual descrito en la Ecuación 3.8: cada repartidor recibe el máximo entre la ganancia por entregas ($p_1 \cdot n_{orders}$) y el pago mínimo garantizado ($p_2 \cdot h_{shift}$). A partir de este modelo se obtienen:

- **Compensación total** ($): suma de las compensaciones individuales de todos los repartidores.
- **Costo por pedido** ($): compensación total dividida entre el número de pedidos entregados.
- **Fracción con compensación mínima**: proporción de repartidores cuya compensación iguala el pago mínimo garantizado, indicando subutilización de la flota.

3. 7.6. Resumen de indicadores

La Tabla 3.4 presenta los 21 indicadores con su definición abreviada y la dimensión a la que pertenecen.

Tabla 3.4: Indicadores clave de desempeño (KPIs) evaluados.

#	Dimensión	Indicador	Definición
1	Calidad	Avg CtD	Media del Click-to-Door (min)
2	Calidad	P50 CtD	Mediana del CtD
3	Calidad	P90 CtD	Percentil 90 del CtD
4	Calidad	P95 CtD	Percentil 95 del CtD
5	Calidad	Avg RtP	Media del Ready-to-Pickup (min)
6	Calidad	P90 RtP	Percentil 90 del RtP
7	Calidad	Avg RtD	Media del Ready-to-Door (min)
8	Calidad	P90 RtD	Percentil 90 del RtD
9	Calidad	SLA Compliance	% de órdenes con CtD ≤ 40 min
10	Calidad	CTD Overage	Exceso promedio sobre τ (min)
11	Cobertura	% Undelivered	Órdenes no entregadas / total
12	Operativa	Orders/C/Hr	Pedidos por repartidor por hora
13	Operativa	Bundles/Hr	Agrupamientos por hora
14	Operativa	Utilización	% del turno conduciendo o en servicio
15	Operativa	Dist. total	Kilómetros recorridos (flota)
16	Operativa	Dist./Pedido	km por pedido entregado
17	Agrup.	Avg Bundle	Tamaño medio de agrupamiento
18	Agrup.	% Multi	% órdenes en agrup. > 1
19	Costo	Comp. total	Compensación total (\$)
20	Costo	Costo/Pedido	Compensación / pedidos entregados (\$)
21	Costo	Frac. Mín.	Fracción de repartidores con pago mínimo

Capítulo 4

Resultados

Este capítulo presenta los resultados obtenidos de la ejecución del simulador bajo cuatro escenarios de saturación de flota, comparando la política base FCFS contra la heurística de horizonte rodante (RH) descrita en el Capítulo 2 e implementada según el Capítulo 3. Cada escenario se ejecuta con diez semillas aleatorias ($s \in \{2025, \ldots, 2034\}$), todas válidas, y los valores reportados corresponden al promedio de las diez réplicas. Los resultados provienen de la corrida `results/experiments/20261023_233115_osrm`; los controles euclidianos (Sección 3. 6.5) corresponden a las corridas `20261023_232220_euclid` (651 m/min) y `20261024_euclid_kappa` (472 m/min), y el análisis de robustez de la Sección 4. 6, a la corrida `20261024_updfirst_osrm`.

4. 1. Configuración experimental

La Tabla 4.1 resume los cuatro escenarios de saturación. Cada semilla genera una instancia sintética de La Paz, B.C.S. con entre 988 y 1 047 órdenes (véase Sección 3.2) y 25 restaurantes. La variable de control es el parámetro `orders_per_courier` (ρ), que determina el tamaño de la flota mediante $|C| = \text{máx}(n_{min}, \lceil n/\rho \rceil)$, donde n es el número de órdenes; por ello la flota real varía ligeramente entre semillas.

Los parámetros del simulador se mantienen constantes según la Tabla 3.2 (§3.1): frecuencia de optimización $f = 5$ min, horizonte $\Delta_u = 20$ min, objetivo SLA $\tau = 40$ min, tiempos de servicio $s_r = s_o = 4$ min y tamaño máximo de agrupamiento $B_{max} = 4$. El enrutamiento se realizó con el binario oficial `osrm-routed` v5.27.1 —la misma versión de la imagen Docker de la Sección 3.1— con el algoritmo *Contraction Hierarchies* sobre los mismos archivos preprocesados

Tabla 4.1: Escenarios de saturación de flota (rango de $|C|$ entre las diez semillas).

ID	Etiqueta	ρ	$\lvert C\rvert$	Saturación
S1	Baja	18	55–59	Baja
S2	Moderada	26	38–41	Moderada
S3	Media	35	29–30	Media
S4	Alta	52	19–21	Alta

del grafo vial de La Paz.

Para cada escenario y semilla se ejecutan las políticas FCFS y RH sobre la misma instancia, generando $4 \times 10 \times 2 = 80$ corridas. Los 21 KPIs de la Tabla 3.4 (§3.7) se calculan para cada corrida. Antes del análisis se verificó la integridad de todas las corridas (`check_results.py`): ninguna orden se recoge antes de su *ready time* ni se entrega antes de recogerse, el número de entregas registradas por los repartidores coincide con el número de órdenes entregadas (no hay entregas dobles), todas las semillas producen entregas y ninguna ruta recurrió al respaldo euclidiano.

4. 2. Calidad de servicio

La Tabla 4.2 presenta los indicadores de calidad de servicio promediados sobre las diez réplicas. El Click-to-Door (CtD) es la métrica principal de calidad percibida por el cliente, definida como el tiempo transcurrido desde que se confirma el pedido hasta que se entrega en el domicilio.

4. 2.1. Análisis del *Click-to-Door*

En el escenario de baja saturación (S1), donde la oferta de repartidores es holgada, ambas políticas tienen un desempeño muy similar: FCFS entrega 1.4 min antes en promedio (28.93 vs 30.31 min), una diferencia estadísticamente significativa pero pequeña (Sección 4. 5), y el cumplimiento SLA es casi idéntico (84.6 % vs 83.4 %). La pequeña ventaja de FCFS se explica porque lleva cada pedido directamente del restaurante al cliente, mientras que RH, incluso con flota holgada, consolida pedidos: cada viaje lleva en promedio 1.93 órdenes y el pedido típico

Tabla 4.2: Indicadores de calidad de servicio: promedio de 10 réplicas. Δ % es el cambio relativo de RH respecto a FCFS; Δpp, la diferencia en puntos porcentuales (calculada con valores sin redondear).

	Avg CtD (min)			P90 CtD (min)			SLA (%)		
Esc.	FCFS	RH	Δ %	FCFS	RH	Δ %	FCFS	RH	Δpp
S1	28.93	30.31	+4.8	43.33	43.80	+1.1	84.6	83.4	−1.1
S2	56.07	35.42	−36.8	97.91	52.87	−46.0	38.1	69.6	+31.5
S3	108.49	45.36	−58.2	169.54	79.10	−53.3	7.3	57.6	+50.3
S4	224.88	73.63	−67.3	321.10	186.18	−42.0	6.2	42.5	+36.3

viaja en un agrupamiento de 2.54 órdenes (media ponderada por pedido), de modo que cada entrega posterior a la primera espera a las anteriores.

A partir de S2 (saturación moderada) la situación cambia de forma marcada. El CtD promedio bajo RH es 36.8 % menor (35.42 vs 56.07 min) y el cumplimiento SLA pasa de 38.1 % a 69.6 % (+31.5 pp). En S3 la reducción del CtD alcanza el 58.2 % (45.36 vs 108.49 min) y el SLA mejora en 50.3 pp; en S4 la reducción es del 67.3 % (73.63 vs 224.88 min), con 36.3 pp más de cumplimiento. El percentil 90 del CtD se reduce entre 42.0 % y 53.3 % en S2–S4, de modo que RH no solo mejora la tendencia central sino que también acorta la cola derecha de la distribución.

La cobertura acentúa la diferencia. Bajo FCFS quedan sin entregar al cierre de la simulación el 11.3 % de los pedidos en S3 y el 39.9 % en S4, frente a 0 % y 0.2 % bajo RH. Como el CtD y el SLA de la Tabla 4.2 se calculan sobre los pedidos entregados, favorecen a FCFS en esos escenarios: medido sobre todas las órdenes generadas, el cumplimiento de FCFS cae a 6.4 % en S3 y a 3.8 % en S4, frente a 57.6 % y 42.4 % bajo RH.

La Figura 4.1 muestra la distribución del Click-to-Door por escenario: las distribuciones de ambas políticas casi coinciden en S1, mientras que en S2–S4 la de FCFS se desplaza y dispersa rápidamente y la de RH permanece compacta.

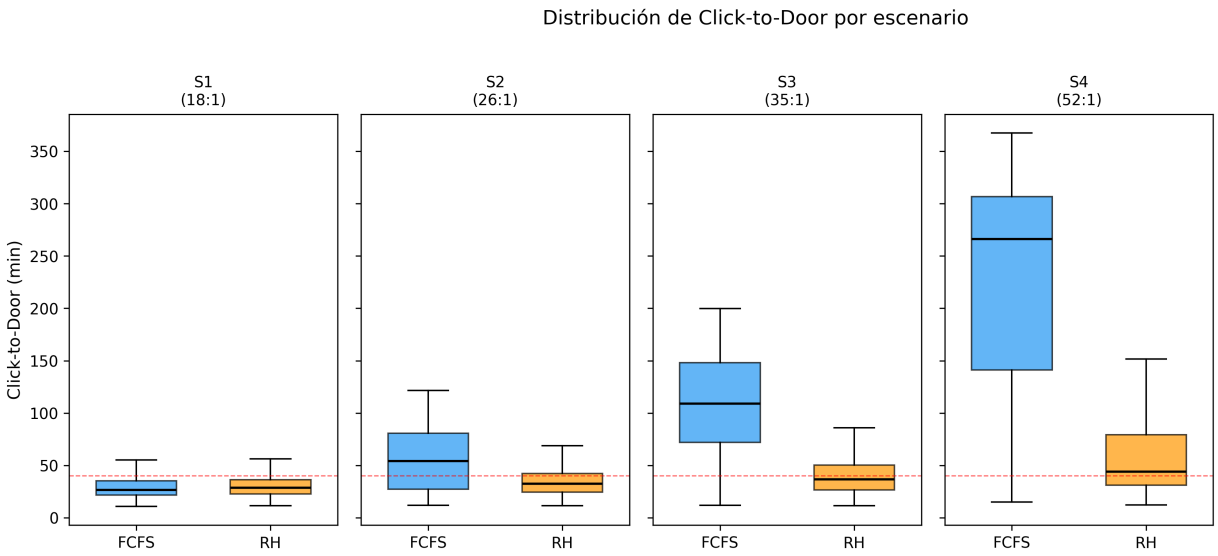


Figura 4.1: Distribución del *Click-to-Door* por escenario y política. La línea roja punteada indica el objetivo SLA de $\tau = 40$ min.

4. 2.2. *Ready-to-Pickup* y exceso sobre el objetivo

La Tabla 4.3 muestra el indicador Ready-to-Pickup (RtP), que mide el tiempo que la comida permanece lista en el restaurante antes de ser recogida, así como el CTD Overage (exceso promedio sobre el objetivo SLA).

Tabla 4.3: *Ready-to-Pickup* y *CTD Overage*: promedio de 10 réplicas.

	Avg RtP [min]			*CTD Overage* [min]		
Esc.	FCFS	RH	Δ %	FCFS	RH	Δ %
S1	7.78	4.94	−36.5	0.82	1.09	–
S2	34.92	9.33	−73.3	21.56	3.86	−82.1
S3	87.35	18.62	−78.7	69.37	12.11	−82.5
S4	203.75	46.20	−77.3	185.64	38.25	−79.4

En todos los escenarios la comida espera menos en el restaurante bajo RH. En S1 el RtP baja de 7.78 a 4.94 min: durante los picos de demanda incluso la flota de S1 queda ocupada por completo, y FCFS, que lleva un pedido por viaje, acumula pedidos listos sin repartidor, mientras que RH recoge varios pedidos de un mismo restaurante en cada visita. Con la saturación, el RtP bajo FCFS crece hasta 203.75 min en S4, reflejo de colas de pedidos listos sin repartidor,

mientras que bajo RH se mantiene en 46.20 min (−77.3 %). El exceso sobre el objetivo de 40 min es despreciable en S1 para ambas políticas (≤ 1.1 min) y se reduce más de un 79 % con RH en S2–S4.

4. 3. Eficiencia operativa

La Tabla 4.4 resume los indicadores de eficiencia operativa.

Tabla 4.4: Indicadores de eficiencia operativa: promedio de 10 réplicas.

	Ord./C/Hr			Utilización (%)			Dist./Pedido (km)		
Esc.	FCFS	RH	Δ %	FCFS	RH	Δpp	FCFS	RH	Δ %
S1	1.43	1.43	0.0	37.7	31.2	−6.5	4.71	4.08	−13.5
S2	2.06	2.06	0.0	55.2	41.4	−13.8	4.88	3.60	−26.2
S3	2.46	2.77	+12.7	66.8	51.4	−15.4	5.07	3.17	−37.4
S4	2.45	4.06	+66.0	67.3	67.8	+0.5	5.20	2.63	−49.4

4. 3.1. Productividad y utilización

Como ambas políticas operan con la misma flota y los mismos turnos, los pedidos por repartidor por hora solo difieren cuando difiere el número de pedidos entregados. En S1 y S2 ambas políticas entregan prácticamente todos los pedidos y la productividad es idéntica (1.43 y 2.06 pedidos/repartidor/hora). El aumento bajo RH en S3 (+12.7 %) y S4 (+66.0 %) refleja la mayor cobertura: RH entrega los pedidos que FCFS deja pendientes al cierre del turno.

La utilización —fracción del turno dedicada a conducir o a dar servicio, sin contar las esperas— es menor bajo RH en S1–S3 (−6.5 a −15.4 pp): los agrupamientos permiten atender la misma demanda con menos kilómetros y menos visitas a restaurantes. Con la flota fija del experimento, esa diferencia se traduce en capacidad ociosa; su valor operativo es que permitiría atender la demanda con menos repartidores (Sección 4. 5.3). En S4 ambas políticas alcanzan una utilización similar (67.3 vs 67.8 %), pero RH la convierte en entregas efectivas mientras que FCFS acumula pedidos sin atender.

4. 3.2. Distancia y enrutamiento

La distancia por pedido es menor bajo RH en todos los escenarios, desde −13.5 % en S1 hasta −49.4 % en S4 (de 5.20 a 2.63 km por pedido). Las rutas con varias entregas reparten la distancia del trayecto al restaurante entre varios pedidos, y la reducción crece con el tamaño de los agrupamientos. Bajo FCFS la distancia por pedido aumenta con la saturación porque los repartidores libres están, en promedio, más lejos del restaurante que los necesita.

4. 4. Agrupamiento y costos

La Tabla 4.5 presenta los indicadores de agrupamiento y el modelo de compensación.

Tabla 4.5: Indicadores de agrupamiento y costos: promedio de 10 réplicas. *Avg Bundle* es el tamaño del agrupamiento en que viaja cada pedido (media ponderada por pedido); Órd./viaje es el número medio de órdenes por ruta.

	Avg Bundle		Órd./viaje	% Multi	Costo/Ped. ($)		
Esc.	FCFS	RH	RH	RH	FCFS	RH	Δ %
S1	1.00	2.54	1.93	74.6	10.80	11.59	+7.4
S2	1.00	2.80	2.17	81.2	10.00	10.03	+0.3
S3	1.00	3.03	2.42	85.6	10.00	10.00	0.0
S4	1.00	3.30	2.80	91.9	10.00	10.00	0.0

4. 4.1. Distribución de agrupamientos

Bajo FCFS cada repartidor recibe órdenes individuales (bundle_size = 1). Bajo RH, el módulo de *bundling* genera agrupamientos de hasta $B_{max} = 4$ órdenes por restaurante y el tamaño promedio crece con la saturación, de 2.54 en S1 a 3.30 en S4 (por viaje, de 1.93 a 2.80 órdenes). La Figura 4.2 muestra la distribución: en S1 solo el 25.4 % de los pedidos viaja de forma individual y el 51.1 % lo hace en agrupamientos de 3 o 4 órdenes; en S4 esa proporción sube al 80.9 %. Que RH consolide pedidos incluso con flota holgada explica su ligera desventaja en tiempo de entrega en S1: el procedimiento de inserción añade un pedido a un agrupamiento existente del mismo restaurante cuando el incremento de costo es menor que el de una ruta

nueva, y el tamaño objetivo Z_t, redondeado hacia arriba, vale 2 en cuanto las órdenes inminentes superan a los repartidores disponibles.

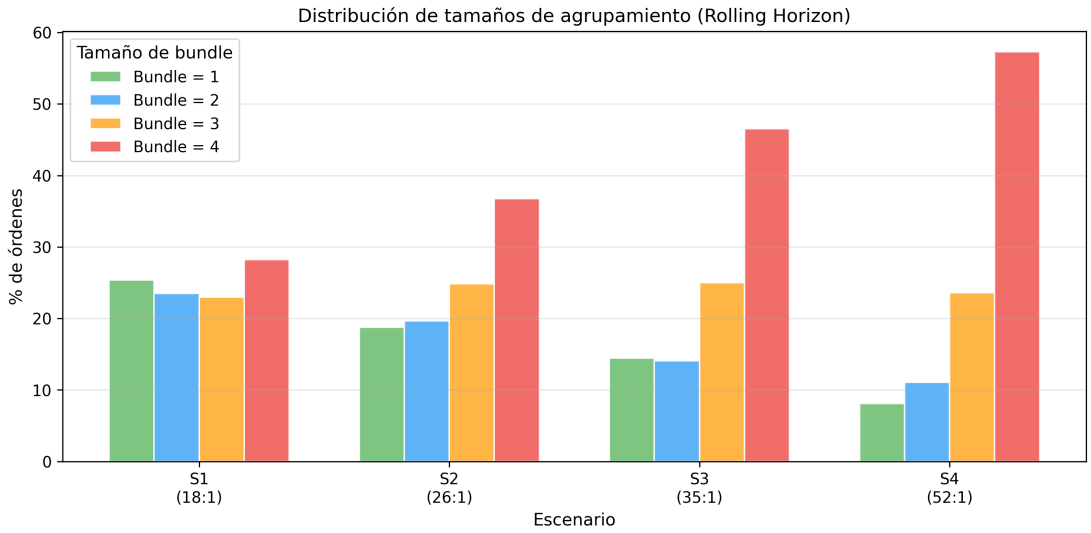


Figura 4.2: Distribución del tamaño de agrupamiento bajo RH por escenario.

4. 4.2. Modelo de compensación

El modelo de compensación asigna $p_1 = \$10$ por pedido entregado con un piso de $p_2 = \$15$/hora de turno. En S1 alrededor del 60 % de los repartidores recibe el pago mínimo bajo ambas políticas. El costo por pedido es mayor bajo RH ($11.59 vs $10.80) porque los agrupamientos concentran las entregas en menos repartidores: los que entregan más cobran por pedido mientras el resto sigue cobrando el piso, lo que eleva el costo total. A partir de S2 el pago variable domina y el costo por pedido converge a $p_1 = \$10$ en ambas políticas. Con estos parámetros de pago, el piso solo es activo en S1, de modo que en S2–S4 el costo por pedido es igual a p_1 por construcción y no discrimina entre políticas.

4. 5. Análisis estadístico

4. 5.1. Diseño inferencial

El experimento posee un diseño de medidas repetidas: cada orden se evalúa bajo ambas políticas con la misma semilla y el mismo mapa. Conforme a este diseño, la prueba primaria es

el test de Wilcoxon signed-rank pareado [33] aplicado a las medias por semilla, de modo que cada semilla aporta una observación independiente al test y se evita la pseudo-replicación que inflaría artificialmente los grados de libertad [32]. Se reporta la diferencia de medias entre semillas con intervalo de confianza t-Student al 95 % y el rank-biserial pareado como tamaño de efecto. Para evaluar si la magnitud del efecto varía entre regímenes se utiliza la prueba ómnibus de Friedman [34] sobre la matriz semillas × escenarios de $\Delta\text{CtD} = \text{CtD}_{\text{FCFS}} - \text{CtD}_{\text{RH}}$. Los cálculos detallados residen en `results/experiments/20261023_233115_osrm/`.

4. 5.2. Resultados

La Tabla 4.6 consolida la inferencia primaria para las métricas clave: Click-to-Door, Ready-to-Door y SLA Compliance. La convención de signos es $\bar{\Delta} = \bar{x}_{\text{FCFS}} - \bar{x}_{\text{RH}}$, por lo que en CtD un $\bar{\Delta} > 0$ favorece a RH (menor tiempo de entrega) y en SLA un $\bar{\Delta} < 0$ favorece a RH (mayor cumplimiento).

Tabla 4.6: Wilcoxon *signed-rank* pareado a nivel semilla ($n = 10$): *Click-to-Door*, *Ready-to-Door* y SLA *Compliance*.

Esc.	Métrica	$\bar{x}_{\text{FCFS}}$	$\bar{x}_{\text{RH}}$	$\bar{\Delta}$ [IC 95 %]	p Wilcoxon	r pareado
S1	CtD (min)	28.93	30.31	−1.38 [−2.35, −0.41]	0.0137	−0.85
S1	RtD (min)	16.95	18.32	−1.38 [−2.35, −0.41]	0.0137	−0.85
S1	SLA (%)	84.6	83.4	+1.1 [−1.8, +4.1]	0.2324	+0.45
S2	CtD (min)	56.07	35.42	+20.64 [+18.98, +22.31]	0.0020	+1.00
S2	RtD (min)	44.08	23.44	+20.64 [+18.98, +22.31]	0.0020	+1.00
S2	SLA (%)	38.1	69.6	−31.5 [−33.7, −29.2]	0.0020	−1.00
S3	CtD (min)	108.49	45.36	+63.13 [+56.14, +70.13]	0.0020	+1.00
S3	RtD (min)	96.52	33.37	+63.15 [+56.16, +70.13]	0.0020	+1.00
S3	SLA (%)	7.3	57.6	−50.3 [−52.6, −48.0]	0.0020	−1.00
S4	CtD (min)	224.88	73.63	+151.24 [+143.71, +158.78]	0.0020	+1.00
S4	RtD (min)	212.92	61.65	+151.27 [+143.73, +158.82]	0.0020	+1.00
S4	SLA (%)	6.2	42.5	−36.3 [−37.2, −35.3]	0.0020	−1.00

El valor $p = 0.0020$ es el mínimo alcanzable por Wilcoxon con $n = 10$ cuando todas las diferencias tienen el mismo signo, condición que se cumple en S2–S4 para las tres métricas.

Con $n = 10$ ese valor solo indica que la diferencia tiene el mismo signo en las diez semillas; la magnitud del efecto se lee en los intervalos de confianza. La diferencia en RtD coincide con la de CtD porque ambas políticas atienden las mismas órdenes con los mismos tiempos de preparación ($\mathrm{RtD} = \mathrm{CtD} - T_{prep}$); solo difiere ligeramente cuando FCFS deja órdenes sin entregar. En S1 la ventaja de FCFS en CtD es estadísticamente significativa pero pequeña (1.38 min, $p = 0.0137$, $r = -0.85$), y la diferencia en SLA no lo es ($p = 0.23$; el intervalo de confianza incluye al cero). No se definió un margen de equivalencia, por lo que no se afirma equivalencia formal entre las políticas en S1. Los resultados se mantienen tras la corrección de Benjamini–Hochberg [35] aplicada a las 24 pruebas a nivel semilla (seis métricas en cuatro escenarios): en S1, $p_{BH} = 0.015$ para CtD y RtD y $p_{BH} = 0.23$ para SLA; en S2–S4, $p_{BH} = 0.002$. La prueba ómnibus de Friedman sobre la matriz 10×4 de ΔCtD confirma que la magnitud del efecto varía entre regímenes de saturación ($\chi^2_F = 30.0$, $p < 0.001$): en cada una de las diez semillas, la ventaja de RH crece de S1 a S4.

4. 5.3. Discusión por regímenes

Los resultados revelan dos regímenes. En **baja saturación (S1)** ambas políticas tienen un desempeño muy similar: FCFS entrega ligeramente antes porque no espera a formar agrupamientos, pero el cumplimiento SLA no difiere y RH reduce el tiempo de espera de la comida y la distancia por pedido. A partir de **saturación moderada (S2)** RH supera decisivamente a FCFS en todas las métricas de servicio: con menos repartidores que pedidos inminentes, FCFS forma colas que crecen sin control, mientras que RH atiende varios pedidos por viaje y mantiene la carga bajo control. El punto de cruce se ubica, por tanto, entre S1 y S2 ($\rho \approx 18$–26 pedidos por repartidor), como se aprecia en la Figura 4.3.

Los escenarios también permiten comparar las políticas con flotas distintas. RH con 29–30 repartidores (S3) obtiene un CtD medio de 45.4 min y un cumplimiento SLA de 57.6 %, mejores que los de FCFS con 38–41 repartidores (S2: 56.1 min y 38.1 %); es decir, RH ofrece mejor servicio con alrededor de 25 % menos repartidores (con la variante de robustez de la Sección 4. 6, el CtD resulta similar y el SLA sigue siendo mayor). La ventaja no se extiende a cualquier reducción: RH con 38–41 repartidores (S2: 35.4 min) no alcanza a FCFS con 55–59 (S1: 28.9 min). Las magnitudes de S3 y S4 corresponden a un régimen en el que FCFS acumula colas que no logra vaciar (hasta 39.9 % de pedidos sin entregar en S4), por lo que su valor es

ilustrativo del colapso de la política reactiva más que de una operación realista.

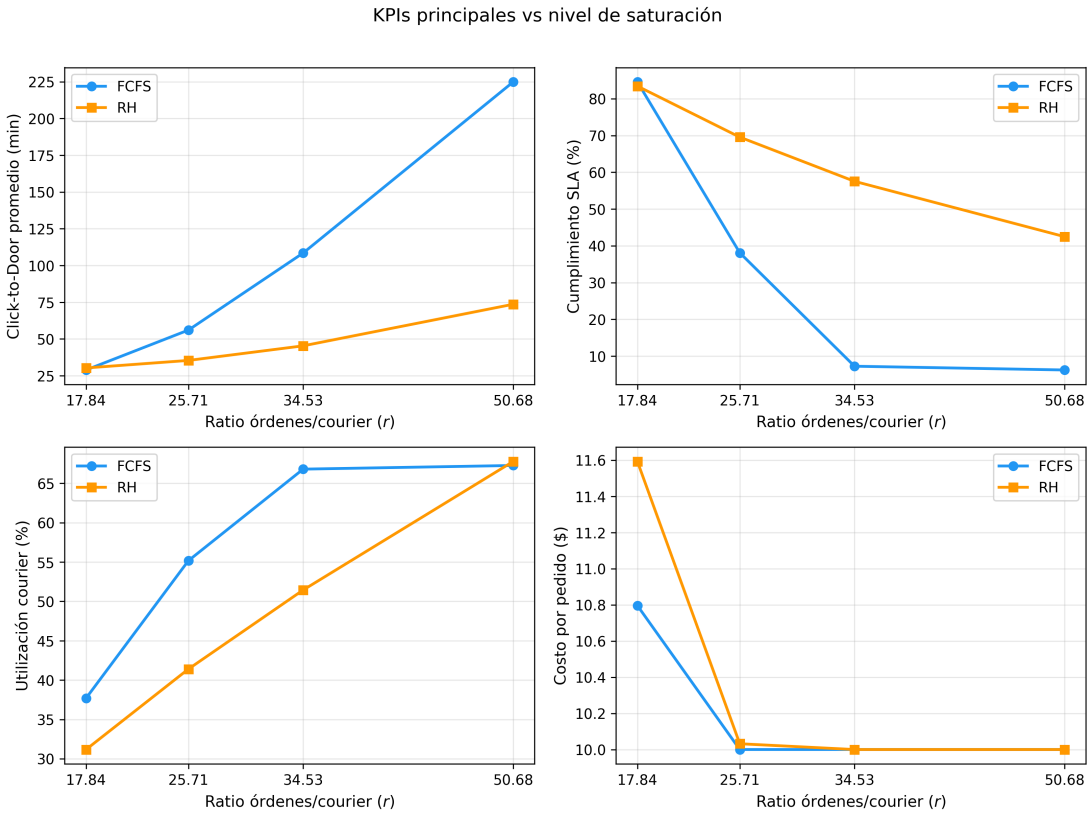


Figura 4.3: Evolución de KPIs principales en función de la saturación. Las curvas de CtD se cruzan entre los ratios 18 y 26.

4. 6. Análisis de robustez: orden de las fases del ciclo

En el motor de simulación la actualización de rutas ocurre después de la asignación (Sección 3.3): un repartidor que termina en el intervalo $(t-f, t]$ recibe su siguiente ruta en la época $t+f$, de modo que espera entre 5 y 10 min. En la campaña principal, el percentil 1 del intervalo entre el fin de un viaje y la siguiente recolección es de 8.4 a 8.9 min en todos los escenarios. Esta espera afecta más a la política que realiza más viajes: FCFS hace un viaje por pedido, mientras que RH lleva en promedio entre 1.93 y 2.80 órdenes por viaje. Para evaluar su efecto se repitió la campaña OSRM completa con la actualización antes de la asignación (`run_experiments.py --update-first`): el repartidor que termina en $(t-f, t]$ recibe ruta en la época t y la espera se reduce a entre 0 y 5 min (percentil 1 de 3.3 a 5.0 min). Las 80 corridas pasan las verificaciones

de integridad. La Tabla 4.7 compara ambas versiones.

Tabla 4.7: Robustez frente al orden de las fases del ciclo: campaña principal (actualización después de asignar) y variante con actualización antes de asignar. Media de 10 semillas; ΔCtD = $\mathrm{CtD_{FCFS}} - \mathrm{CtD_{RH}}$ (min), cambio relativo del CtD de RH respecto a FCFS (%, media por semilla), $\Delta\mathrm{SLA} = \mathrm{SLA_{RH}} - \mathrm{SLA_{FCFS}}$ (pp) y porcentaje de pedidos que FCFS no entrega.

	ΔCtD (min)		Cambio rel. (%)		ΔSLA (pp)		No entreg. FCFS (%)	
Esc.	Princ.	Variante	Princ.	Variante	Princ.	Variante	Princ.	Variante
S1	−1.38	−6.77	+5.0	+29.9	−1.1	−14.7	0.0	0.0
S2	20.64	5.40	−36.7	−14.6	+31.5	+19.6	0.1	0.0
S3	63.13	26.27	−57.9	−41.4	+50.3	+34.6	11.3	0.4
S4	151.24	97.52	−67.2	−63.0	+36.3	+45.1	39.9	23.9

Con la espera reducida ambas políticas mejoran, y FCFS mucho más que RH: en S2 el CtD medio de FCFS baja de 56.1 a 36.4 min y el de RH, de 35.4 a 31.0 min. Las conclusiones cualitativas se mantienen. En S1, FCFS sigue siendo mejor en las diez semillas, ahora con una ventaja mayor (6.8 min) y con una diferencia en SLA que pasa a ser significativa (100 % frente a 85.3 %, $p = 0.002$). En S2–S4, RH sigue siendo mejor en las diez semillas de cada escenario ($p = 0.002$ en CtD y SLA), y el punto de cruce permanece entre S1 y S2. Las magnitudes, en cambio, son menores: la reducción del CtD es de 14.6 % en S2, 41.4 % en S3 y 63.0 % en S4, frente a 36.7 %, 57.9 % y 67.2 % en la campaña principal. También se modera la comparación con flotas distintas de la Sección 4. 5.3: con la variante, RH con 29–30 repartidores (S3) obtiene un CtD similar al de FCFS con 38–41 (S2; 37.0 frente a 36.4 min, $p = 0.23$) y un cumplimiento SLA mayor (67.8 frente a 61.5 %, $p = 0.03$). Por tanto, el signo de los resultados es robusto a la convención del ciclo, mientras que las magnitudes de la campaña principal deben leerse como una cota superior de la ventaja de RH.

4. 7. Efecto de la topología vial

4. 7.1. Circuidad de la red de La Paz

Sobre los 10 184 trayectos restaurante–cliente de las diez instancias, la distancia por la red vial OSRM es en promedio 3.30 km frente a 2.39 km en línea recta: un índice de circuidad agregado de $\kappa = 1.38$ (mediana 1.37; percentiles 10 y 90 de 1.19 y 1.61). Es decir, los trayectos reales son un 38 % más largos que la estimación euclidiana, valor dentro del rango típico de 1.2 a 1.4 que recogen Boyacı et al. [12]. La velocidad media que OSRM asigna a esos trayectos es de 39.1 km/h (651 m/min).

Conviene precisar el alcance geográfico de esta medición. El polígono de muestreo (Sección 3.2) abarca 8.4 km^2 del centro urbano y su lado noroeste sigue la costa, por lo que ningún trayecto rodea la ensenada: el percentil 99 de κ es 1.90 y solo el 0.5 % de los pares supera $\kappa = 2$. La circuidad medida refleja, por tanto, la retícula vial y los sentidos de circulación del centro, no los rodeos por la bahía descritos en la Sección 1.2. Además, 13 de los 10 184 pares tienen $\kappa < 1$, lo que solo es posible si OSRM ajusta (*snapping*) alguno de los puntos a una vía distante; su efecto sobre los promedios es despreciable.

4. 7.2. Comparación con el escenario de control euclidiano

La Tabla 4.8 compara la ventaja de RH sobre FCFS en el escenario principal (OSRM) y en el control euclidiano, que usa las mismas instancias y la misma velocidad media pero trayectos en línea recta (Sección 3. 6.5). Las comparaciones se hacen semilla por semilla con la prueba de Wilcoxon pareada ($n = 10$); el cambio relativo se calcula por semilla y luego se promedia, por lo que puede diferir ligeramente del cociente de las medias de la Tabla 4.2 (por ejemplo, +5.0 frente a +4.8 % en S1).

Con la red vial real, la ventaja de RH en minutos es mayor en los cuatro escenarios ($p = 0.002$ en todos): en S1 la desventaja de RH se reduce de 4.5 a 1.4 min y en S3 la ventaja pasa de 37.1 a 63.1 min. En términos relativos la ventaja también es mayor en S1–S3, mientras que en S4 es estadísticamente igual (-67.2 vs -68.3 %, $p = 0.08$). En cumplimiento SLA el efecto es mixto: mayor con OSRM en S1 y S3, igual en S2 y menor en S4, donde ambas políticas ya incumplen el objetivo en la mayoría de los pedidos.

Tabla 4.8: Ventaja de RH sobre FCFS con la red vial real (OSRM) y con distancias en línea recta (Euclid.): media de 10 semillas. $\Delta\text{CtD} = \text{CtD}_{\text{FCFS}} - \text{CtD}_{\text{RH}}$; $\Delta\text{SLA} = \text{SLA}_{\text{RH}} - \text{SLA}_{\text{FCFS}}$.

	ΔCtD (min)			Cambio relativo del CtD (%)			ΔSLA (pp)		
Esc.	OSRM	Euclid.	p	OSRM	Euclid.	p	OSRM	Euclid.	p
S1	−1.38	−4.53	0.002	+5.0	+19.8	0.002	−1.1	−9.5	0.002
S2	20.64	11.50	0.002	−36.7	−27.4	0.002	+31.5	+31.4	0.922
S3	63.13	37.09	0.002	−57.9	−49.9	0.002	+50.3	+41.5	0.002
S4	151.24	123.30	0.002	−67.2	−68.3	0.084	+36.3	+47.1	0.002

El mecanismo principal es la carga efectiva: los rodeos que impone la red alargan cada viaje en torno a un 38 %, de modo que la misma flota enfrenta una saturación mayor, y la ventaja de RH crece con la saturación. Así, el uso de distancias euclidianas subestima los tiempos de entrega de ambas políticas (el CtD de FCFS en S2 pasa de 41.8 min en el control a 56.1 min con OSRM; el de RH, de 30.3 a 35.4 min) y también el beneficio de coordinar los despachos en buena parte de los escenarios. Para RH, pasar del control a la red real aumenta el CtD medio un 10 % en S1 y un 29 % en S4.

Para separar el efecto de la longitud de los viajes del de la estructura de la red, se repitió el control euclidiano con la velocidad reducida en el factor de circuidad ($651/1.38 \approx 472$ m/min), de modo que los trayectos en línea recta duran en promedio lo mismo que por la red. La Tabla 4.9 muestra que este ajuste escalar explica entre el 62 y el 84 % de la diferencia entre el control y la red real en la ventaja de RH. El resto, aún significativo ($p \leq 0.004$ en los cuatro escenarios), corresponde a la estructura de la red: la variación de la circuidad entre pares, la asimetría de los sentidos de circulación y los tramos de acceso de cada repartidor al restaurante. Parte de ese resto puede deberse también a que FCFS elige al repartidor por distancia en línea recta, criterio que con la red real no siempre identifica al que llega primero (Sección 3.4).

4. 8. Desempeño computacional

Para sustentar el requisito de hardware de consumo (Sección 1.6) se midió el tiempo de cómputo con la semilla 2025 (1 038 órdenes), ejecutando cada escenario en un proceso independiente, en secuencia y sin otras cargas, sobre una computadora portátil Apple M1 Pro con

Tabla 4.9: Ventaja de RH en CtD ($\Delta\text{CtD} = \text{CtD}_{\text{FCFS}} - \text{CtD}_{\text{RH}}$, min) con la red vial real, con el control euclidiano a la velocidad media (651 m/min) y con el control euclidiano escalado por κ (472 m/min): media de 10 semillas. La última columna es la fracción de la diferencia OSRM–control explicada por el escalamiento, (Euclid.$\times\kappa$ − Euclid.)/(OSRM − Euclid.).

Esc.	OSRM	Euclid.	Euclid.$\times\kappa$	p (OSRM vs. $\times\kappa$)	Explicado por κ
S1	−1.38	−4.53	−2.46	0.002	66 %
S2	20.64	11.50	17.83	0.002	69 %
S3	63.13	37.09	53.17	0.002	62 %
S4	151.24	123.30	146.70	0.004	84 %

16 GB de RAM, Python 3.9.6 y el servidor `osrm-routed` v5.27.1 local. La Tabla 4.10 resume los resultados: simular un día de operación (unas 150 épocas de $f = 5$ min) toma entre 1.8 y 2.7 s con FCFS y entre 57 y 80 s con RH, incluidas las consultas a OSRM, es decir, menos de 0.6 s por época en promedio para RH. La memoria máxima de cada proceso, que ejecuta ambas políticas y el cálculo de indicadores, fue de 194 a 221 MB, muy por debajo de los 8 GB considerados en el alcance.

Tabla 4.10: Tiempo de cómputo por día simulado (semilla 2025, 1 038 órdenes) y memoria máxima del proceso, con OSRM local.

Esc.	$\lvert C\rvert$	FCFS (s)	RH (s)	Memoria máx. (MB)
S1	58	2.7	74.4	213
S2	40	2.7	57.0	194
S3	30	2.4	58.7	195
S4	20	1.8	79.9	221

4. 9. Comparación con la literatura y limitaciones

4. 9.1. Comparación con la literatura

Reyes et al. [13] reportan que su heurística de horizonte rodante obtiene, en instancias pequeñas resueltas a optimalidad, un CtD promedio apenas un 4 % superior al óptimo, pero no

publican una comparación directa contra una política FCFS equivalente, por lo que no es posible contrastar cuantitativamente la magnitud de las mejoras. Cualitativamente, los resultados coinciden con su interpretación del agrupamiento como un recurso para mitigar la escasez de capacidad: su tamaño objetivo busca agrupamientos de una sola orden cuando hay menos órdenes que repartidores y agrupamientos mayores cuando el sistema está bajo presión. En la implementación de este trabajo, sin embargo, RH consolida pedidos incluso en S1 (1.93 órdenes por viaje; el pedido típico viaja en un agrupamiento de 2.54), más de lo que esa intención sugiere, lo que explica su ligera desventaja en baja saturación. La circuidad medida (1.38) queda dentro del rango típico de 1.2 a 1.4 que recogen Boyacı et al. [12], y el control escalado por κ muestra que, en estas instancias, la mayor parte del efecto de la red vial sobre la comparación de políticas se debe a la longitud de los viajes, como supone el factor de conversión γ de Reyes et al. [13], y una parte menor a su estructura.

4. 9.2. Limitaciones de los resultados

Los resultados deben interpretarse considerando las limitaciones del modelo:

- La demanda es sintética: sigue un proceso de Poisson con los picos de recolección de paquetes de LaDe (09:00 y 17:00), que no corresponden a la demanda de comida ni a los horarios de comida locales, y las ubicaciones se muestrean de forma uniforme en un polígono de 8.4 km^2 del centro urbano, con 25 restaurantes (§1.6, §3.2).

- Los tiempos de preparación son homogéneos ($T_{prep} \sim \text{máx}(5, \mathcal{N}(12, 3^2))$ min) y los tiempos de servicio constantes ($s_r = s_o = 4$ min).

- La red vial se asume estática (sin congestión dinámica) y los tiempos provienen del perfil `car.lua` de OSRM, que corresponde a automóviles y no a motocicletas o bicicletas.

- Todos los repartidores comparten un único turno que cubre la jornada completa.

- El reloj avanza en épocas de $f = 5$ min y la actualización de rutas ocurre después de la asignación, de modo que un repartidor que termina su ruta espera entre 5 y 10 min su siguiente asignación, aunque los instantes de recolección y entrega se registran con precisión de segundos. El efecto de esta convención se evalúa en la Sección 4. 6.

- El CtD y el SLA de las tablas principales se calculan sobre los pedidos entregados; en S3 y S4 los pedidos que FCFS no entrega no aportan tiempos, por lo que esas medias subestiman el deterioro de FCFS (el SLA sobre todas las órdenes se reporta en la Sección 4.2).
- Con los parámetros de pago adoptados ($p_1 = \$10$ por pedido, $p_2 = \$15/\text{h}$), el pago mínimo solo es activo en S1, por lo que los indicadores de costo no discriminan entre políticas en S2–S4.
- Los controles euclidianos usan una única velocidad para toda la red, por lo que aíslan el efecto de la geometría de las rutas pero no el de las diferencias de velocidad entre tipos de vía.

Capítulo 5

Conclusiones

Este capítulo responde las preguntas de investigación planteadas en la Sección 1. 2.4, verifica el cumplimiento de los objetivos y de la hipótesis, resume las contribuciones y propone líneas de investigación futura.

5. 1. Respuesta a las preguntas de investigación

Pregunta central. En las instancias evaluadas del centro urbano de La Paz, coordinar los despachos no compensa cuando la flota es holgada, pero sí a partir de una saturación moderada: con 18 pedidos por repartidor FCFS entrega 1.4 min antes que RH, y con 26 o más pedidos por repartidor RH reduce el tiempo de entrega y mejora el cumplimiento del servicio en todas las semillas. Usar distancias euclidianas en lugar de la red vial real subestima los tiempos de entrega de ambas políticas y la ventaja de coordinar.

PI1 (saturación). La ventaja de RH crece con la saturación de la flota (Secciones 4.2 y 4.5). El punto de cruce se ubica entre 18 y 26 pedidos por repartidor; por encima de él RH reduce el CtD medio entre 15 y 37 % en S2 y entre 63 y 67 % en S4, según la convención del ciclo de simulación (Sección 4. 6), y entrega prácticamente todos los pedidos cuando FCFS deja de atender una parte de ellos.

PI2 (red vial). Los trayectos restaurante–cliente por la red vial del centro de La Paz son en promedio un 38 % más largos que en línea recta ($\kappa = 1.38$). Con la red real, la ventaja de

RH en minutos es mayor que con distancias euclidianas en los cuatro escenarios; entre el 62 y el 84 % de esa diferencia se explica por la mayor longitud media de los viajes y el resto por la estructura de la red (Sección 4. 7).

PI3 (reproducibilidad y recursos). Sí. Toda la cadena —OpenStreetMap, OSRM, Python— es de código abierto, las instancias se regeneran a partir de semillas fijas, cada corrida pasa verificaciones automáticas de integridad, y simular un día de operación con RH toma alrededor de un minuto en una computadora portátil, con menos de 250 MB de memoria (Sección 4. 8).

5. 2. Verificación de objetivos

5. 2.1. Objetivo general

El objetivo general —evaluar la eficiencia operativa de la política de despacho *Rolling Horizon* frente a FCFS mediante un entorno de simulación reproducible con restricciones topológicas reales de La Paz, B.C.S.— se cumple. Se construyó el entorno y con él se ejecutaron 80 corridas pareadas válidas (4 escenarios $\times$ 10 semillas $\times$ 2 políticas) sobre la red vial real, más dos controles euclidianos de 80 corridas cada uno. La evaluación muestra que la ventaja de RH depende de la saturación de la flota: con 18 pedidos por repartidor, FCFS entrega 1.4 min antes; a partir de 26 pedidos por repartidor, RH reduce el CtD medio entre 37 % y 67 % y entrega prácticamente todos los pedidos. Estos resultados valen para instancias sintéticas del centro urbano de La Paz (8.4 km^2), con 25 restaurantes, turno único y red sin congestión.

5. 2.2. Objetivos específicos

OE 1: Generación de instancias sintéticas. Se implementó un generador estocástico (`make_synth_ord`
que proyecta sobre el área urbana de La Paz el perfil diario de LaDe (picos a las 9:00 y a las 17:00). Las instancias no están calibradas con datos locales, que no se tienen disponibles: sus parámetros se fijan con valores de la literatura (Secciones 2.4 y 3.2). La validez geoespacial se garantiza al muestrear dentro de un polígono terrestre del centro urbano. Las diez semillas ($s \in \{2025, \ldots, 2034\}$) producen instancias reproducibles de 988 a 1 047 órdenes distribuidas en 25 restaurantes, y todas generaron resultados válidos.

OE 2: Integración OSRM–simulador. El motor de simulación se integró con OSRM v5.27.1 sobre cartografía OpenStreetMap, utilizando *Contraction Hierarchies* para el cálculo de tiempos y rutas. El 100 % de las rutas de la campaña principal se obtuvo de OSRM, sin recurrir al respaldo euclidiano. La diferencia frente a las distancias euclidianas se cuantificó con el índice de circuidad ($\kappa = 1.38$) y con dos escenarios de control. La exactitud de los tiempos de OSRM frente a una referencia externa (por ejemplo, trazas GPS de repartidores) no se evaluó.

OE 3: Implementación de las políticas. Se implementaron ambas políticas (FCFS y RH) con instrumentación completa de métricas y con la temporización de Reyes et al. [13]: espera en el restaurante hasta que la comida está lista y tiempos de servicio en recolección y entrega. El módulo de *bundling* agrupa hasta $B_{max} = 4$ órdenes por restaurante. Se verificó la integridad de todas las corridas: ninguna orden se entrega antes de estar lista, cada orden pertenece a un solo agrupamiento y ninguna se entrega dos veces. Estas verificaciones comprueban la consistencia de la simulación; la fidelidad de la implementación de RH frente a la de Reyes et al. [13] se sustenta en el seguimiento de su descripción, con las adaptaciones documentadas en el Capítulo 3.

OE 4: Cuantificación estadística. Se aplicó la prueba de Wilcoxon *signed-rank* pareada a nivel semilla ($n = 10$) sobre *Click-to-Door*, *Ready-to-Door* y SLA *Compliance* en los cuatro escenarios, con corrección de Benjamini–Hochberg. En S2–S4 todas las pruebas arrojan $p = 0.0020$ —el mínimo alcanzable con $n = 10$, que indica que las diez semillas coinciden en el signo— a favor de RH, con intervalos de confianza alejados del cero. En S1, FCFS obtiene un CtD y un RtD 1.38 min menores ($p = 0.0137$) y la diferencia en SLA no es significativa ($p = 0.23$). La prueba de Friedman ($\chi^2_F = 30.0$, $p < 0.001$) confirma que la magnitud del efecto varía entre regímenes de saturación.

5. 3. Verificación de la hipótesis

La hipótesis planteaba que la política RH superaría significativamente a FCFS en eficiencia operativa (órdenes por hora) y nivel de servicio (*Ready-to-Door*). Formulada de manera general, **la hipótesis no se verifica**, porque no se cumple en baja saturación; los datos sí sostienen una versión condicionada al nivel de saturación:

- **Baja saturación (S1).** FCFS entrega 1.4 min antes en promedio (28.93 vs 30.31 min; IC 95 % de la diferencia [0.41, 2.35] min) y su *Ready-to-Door* también es 1.4 min menor; el cumplimiento SLA no difiere significativamente. RH sí reduce el tiempo de espera de la comida (−36.5 %) y la distancia por pedido (−13.5 %).

- **Saturación moderada a alta (S2–S4).** RH reduce el CtD promedio entre 36.8 % y 67.3 %, el *Ready-to-Door* entre 46.8 % y 71.0 %, y mejora el cumplimiento SLA entre 31.5 y 50.3 pp, en las diez semillas de cada escenario ($p = 0.002$). En S3 y S4, además, entrega prácticamente todos los pedidos, mientras que FCFS deja sin entregar el 11.3 % y el 39.9 %.

El componente de eficiencia operativa de la hipótesis, medido como órdenes por repartidor por hora, no permite discriminar entre políticas con este diseño: como ambas operan con la misma flota y los mismos turnos, esa métrica solo difiere cuando difiere la cobertura. Por eso es idéntica en S1–S2, y su aumento bajo RH en S3 y S4 (+12.7 y +66.0 %) refleja los pedidos que FCFS no entrega, no una mayor productividad por repartidor. La eficiencia de RH se manifiesta, en cambio, en la distancia por pedido, menor en todos los escenarios (−13.5 % a −49.4 %), y en la flota necesaria para un nivel de servicio dado: con alrededor de 25 % menos repartidores (S3), RH ofrece un servicio al menos comparable al de FCFS en S2 (Secciones 4. 5.3 y 4. 6). La forma condicional de la hipótesis es, por tanto, un hallazgo del estudio y no la hipótesis original. El análisis de robustez (Sección 4. 6) confirma estas conclusiones cualitativas cuando el repartidor recibe ruta en la misma época en que termina la anterior: FCFS sigue siendo mejor en S1 y RH en S2–S4, en todas las semillas, aunque la ventaja de RH es menor (por ejemplo, una reducción del CtD de 14.6 % en S2 en lugar de 36.7 %), por lo que las magnitudes reportadas son una cota superior.

5. 4. Contribuciones principales

1. **Entorno de simulación reproducible.** Se desarrolló un simulador de código abierto que integra OSRM, cartografía OSM y generación estocástica de demanda, con verificaciones automáticas de integridad de los resultados, aplicado a una ciudad latinoamericana de escala media.

2. **Evidencia empírica del efecto de la saturación.** Se documentó cuantitativamente que, en la configuración evaluada, RH supera a FCFS a partir de saturación moderada y que el punto de cruce se ubica entre 18 y 26 pedidos por repartidor. La desventaja de RH en S1 se asocia con el redondeo hacia arriba del tamaño objetivo de agrupamiento, por lo que el umbral es una propiedad de la implementación y de las instancias evaluadas, y no una regla general para operadores.

3. **Magnitud de la mejora y ahorro de flota.** RH con alrededor de 25 % menos repartidores (S3) ofrece un servicio al menos comparable al de FCFS en S2: CtD y SLA mejores en la campaña principal, y CtD similar con SLA mayor en la variante de robustez. En el escenario más demandante (S4, $\rho = 52$), RH reduce el CtD promedio entre 63 % y 67 % según la convención del ciclo y entrega prácticamente todos los pedidos, frente al 60–76 % de FCFS; esa magnitud ilustra el colapso de la política reactiva bajo saturación extrema.

4. **Medición del efecto de la red vial.** Se midió que los trayectos por la red vial del centro de La Paz son un 38 % más largos que en línea recta ($\kappa = 1.38$) y que ignorar esta diferencia subestima los tiempos de entrega de ambas políticas y la carga efectiva de la flota. Un control euclidiano escalado por κ mostró que entre el 62 y el 84 % del efecto de la red sobre la ventaja de RH se debe a la longitud de los viajes y el resto a la estructura de la red.

5. **Arquitectura exclusivamente open-source.** Toda la cadena — OSRM, OSM, Python, Docker — se basa en herramientas de código abierto, demostrando la viabilidad de soluciones de optimización logística sin dependencia de software propietario.

5. 5. Trabajo futuro

- **Punto de cruce fino.** Explorar valores de ρ entre 18 y 26 para localizar con mayor precisión el punto a partir del cual RH supera a FCFS.

- **Latencia de RH con flota holgada.** Evaluar reglas que eviten agrupar cuando sobran repartidores (por ejemplo, un tamaño objetivo Z_t redondeado al entero más cercano o una frecuencia de optimización menor), para eliminar la desventaja de RH en baja saturación.

- **Ablación y líneas base más fuertes.** Aislar la contribución de cada mecanismo de RH (agrupamiento, asignación global, compromiso en dos etapas, grupos de prioridad), como en las variantes de Reyes et al. [13], y comparar RH con políticas reactivas que también agrupen o que elijan al repartidor con tiempos de la red vial.

- **Validación con instancias de referencia.** Ejecutar la implementación sobre las instancias públicas de Reyes et al. [13] para contrastar sus resultados con los publicados.

- **Demanda real.** Sustituir las instancias sintéticas por datos operacionales de una plataforma de *delivery* activa en La Paz, con los horarios de comida y la distribución espacial de la demanda locales.

- **Cobertura geográfica y calidad de la red.** Ampliar el polígono de muestreo al área urbana completa —incluidos los trayectos que rodean la ensenada—, medir la conectividad del grafo vial y validar los tiempos de OSRM contra trazas reales de repartidores.

- **Congestión dinámica.** Incorporar perfiles de velocidad variables por hora del día en OSRM (*traffic profiles*) para capturar el efecto de la congestión sobre los tiempos de viaje.

- **Calibración de pesos.** Explorar métodos de ajuste automático de los parámetros de la función de costo y del peso de asignación (β, θ) mediante búsqueda bayesiana o aprendizaje por refuerzo.

- **Flota heterogénea y turnos.** Modelar vehículos con diferentes velocidades y capacidades (bicicletas, motocicletas, automóviles) y turnos escalonados de repartidores, para evaluar su impacto en la formación de agrupamientos.

Bibliografía

[1] Jan Karel Lenstra and Alexander H. G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2):221–227, 1981. doi: 10.1002/net.3230110211.

[2] G. B. Dantzig and J. H. Ramser. The truck dispatching problem. *Management Science*, 6 (1):80–91, 1959. doi: 10.1287/mnsc.6.1.80.

[3] Jose Cáceres-Cruz, Pol Arias, Daniel Guimarans, Daniel Riera, and Angel A. Juan. Rich vehicle routing problem: Survey. *ACM Computing Surveys*, 47(2):32:1–32:28, December 2014. doi: 10.1145/2666003.

[4] Gilbert Laporte. Fifty years of vehicle routing. *Transportation Science*, 43(4):408–416, 2009. doi: 10.1287/trsc.1090.0301.

[5] Reza Jazemi, Ensieh Alidadiani, Kwangseog Ahn, and Jaejin Jang. A review of literature on vehicle routing problems of last-mile delivery in urban areas. *Applied Sciences*, 13(24): 13015, 2023. doi: 10.3390/app132413015.

[6] Harilaos N. Psaraftis. A dynamic programming solution to the single vehicle many-to-many immediate request dial-a-ride problem. *Transportation Science*, 14(2):130–154, 1980. doi: 10.1287/trsc.14.2.130.

[7] Jean-François Cordeau and Gilbert Laporte. The dial-a-ride problem: Models and algorithms. *Annals of Operations Research*, 153:29–46, 2007. doi: 10.1007/s10479-007-0170-8.

[8] Soumia Ichoua, Michel Gendreau, and Jean-Yves Potvin. Exploiting knowledge about future demands for real-time vehicle dispatching. *Transportation Science*, 40(2):211–225, 2006. doi: 10.1287/trsc.1050.0114.

[9] Allan Larsen. *The Dynamic Vehicle Routing Problem.* Tesis doctoral, Department of Mathematical Modelling (IMM), Technical University of Denmark, Lyngby, Denmark, 2000. IMM-PHD-2000-73.

[10] Victor Pillac, Michel Gendreau, Christelle Guéret, and Andrés L. Medaglia. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1):1–11, 2013. doi: 10.1016/j.ejor.2012.08.015.

[11] Rafael H. M. Pereira, Marcus Saraiva, Daniel Herszenhut, Carlos Kaue Braga, and Matthew Wigginton Conway. r5r: Rapid realistic routing on multimodal transport networks with R5 in R. *Findings*, 2021. doi: 10.32866/001c.21262. URL `https://findingspress.org/article/21262-r5r-rapid-realistic-routing-on-multimodal-transport-networks-with-r5-in-r`.

[12] Burak Boyacı, Thu Huong Dang, and Adam N. Letchford. Vehicle routing on road networks: How good is euclidean approximation? *Computers & Operations Research*, 129:105197, 2021. doi: 10.1016/j.cor.2020.105197.

[13] Damian Reyes, Alan Erera, Martin Savelsbergh, Sagar Sahasrabudhe, and Ryan O'Neil. The meal delivery routing problem. Preprint, Optimization Online, March 2018. URL `http://www.optimization-online.org/DB_FILE/2018/03/6512.pdf`.

[14] Barış Yıldız and Martin Savelsbergh. Provably high-quality solutions for the meal delivery routing problem. *Transportation Science*, 53(5):1372–1388, 2019. doi: 10.1287/trsc.2018.0887.

[15] Miguel Andres Figliozzi. Analysis of the efficiency of urban commercial vehicle tours: Data collection, methodology, and policy implications. *Transportation Research Part B: Methodological*, 41(9):1014–1032, 2007. doi: 10.1016/j.trb.2007.04.006.

[16] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014. URL `http://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment`.

[17] Kaixiang Lin, Renyu Zhao, Zhe Xu, and Jiayu Zhou. Efficient large-scale fleet management via multi-agent deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD*

International Conference on Knowledge Discovery & Data Mining (KDD '18), pages 1774–1783, 2018. doi: 10.1145/3219819.3219993.

[18] Lixia Wu, Haomin Wen, Haoyuan Hu, Xiaowei Mao, Yutong Xia, Ergang Shan, Jianbin Zheng, Junhong Lou, Yuxuan Liang, Liuqing Yang, Roger Zimmermann, Youfang Lin, and Huaiyu Wan. LaDe: The first comprehensive last-mile express dataset from industry. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, pages 5991–6002, 2024. doi: 10.1145/3637528.3671548.

[19] Andrés Muñoz-Villamizar, Elyn L. Solano-Charris, Lorena Reyes-Rubiano, and Javier Faulin. Measuring disruptions in last-mile delivery operations. *Logistics*, 5(1):17, 2021. doi: 10.3390/logistics5010017.

[20] Marlin W. Ulmer, Barrett W. Thomas, Ann M. Campbell, and Nicholas Woyak. The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times. *Transportation Science*, 55(1):75–100, 2021. doi: 10.1287/trsc.2020.1000.

[21] Zachary Steever, Mark Karwan, and Chase Murray. Dynamic courier routing for a food delivery service. *Computers & Operations Research*, 107:173–188, 2019. doi: 10.1016/j.cor.2019.03.008.

[22] Marlin W. Ulmer, Ninja Soeffker, and Dirk C. Mattfeld. Value function approximation for dynamic multi-period vehicle routing. *European Journal of Operational Research*, 269(3):883–899, 2018. doi: 10.1016/j.ejor.2018.02.038.

[23] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109.

[24] Mordechai Haklay. How good is volunteered geographical information? a comparative study of openstreetmap and ordnance survey datasets. *Environment and Planning B: Planning and Design*, 37(4):682–703, 2010. doi: 10.1068/b35097.

[25] Geoff Boeing. Street network models and indicators for every urban area in the world. *Geographical Analysis*, 54(3):519–535, 2022. doi: 10.1111/gean.12281.

[26] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[27] Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Proceedings of the 7th International Workshop on Experimental Algorithms (WEA 2008)*, volume 5038 of *Lecture Notes in Computer Science*, pages 319–333, Berlin, Heidelberg, 2008. Springer. doi: 10.1007/978-3-540-68552-4_24.

[28] Dennis Luxen and Christian Vetter. Real-time routing with OpenStreetMap data. In *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, GIS '11, pages 513–516, Chicago, IL, USA, 2011. ACM. doi: 10.1145/2093973.2094062.

[29] Sheldon M. Ross. *Introduction to Probability Models*. Academic Press, Amsterdam, 11th edition, 2014. ISBN 978-0-12-407948-9.

[30] Florentin D. Hildebrandt and Marlin W. Ulmer. Supervised learning for arrival time estimations in restaurant meal delivery. *Transportation Science*, 56(4):1058–1084, 2022. doi: 10.1287/trsc.2021.1095.

[31] Open Source Routing Machine. Open Source Routing Machine – documentación y manual de usuario. `https://github.com/Project-OSRM/osrm-backend/wiki`, 2025. Consultado en mayo de 2025.

[32] Stuart H. Hurlbert. Pseudoreplication and the design of ecological field experiments. *Ecological Monographs*, 54(2):187–211, 1984. doi: 10.2307/1942661.

[33] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6): 80–83, 1945. doi: 10.2307/3001968.

[34] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937. doi: 10.1080/01621459.1937.10503522.

[35] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.